sim.py (81줄)

```
#!/usr/bin/env python3
"""RISC-V(RV32I 부분집합) 기능 시뮬레이터 실행 진입점.

사용법:  python sim.py <text.bin 경로> <data.bin 경로>

정상 종료(invalid instruction 을 fetch)
  - 현재 작업 디렉터리에 data.out (0x10000000 부터 정확히 1,024 바이트) 생성
  - 표준 출력에 한 줄:  PC 0x%08x invalid instruction 0x%08x
종료 코드
  0 정상 종료 / 1 인수·파일 입출력 오류 / 2 실행 한도 초과 / 3 메모리 범위·정렬 위반
진단 메시지는 모두 표준 오류(stderr)로만 출력한다.
"""

import sys

from rvsim.cpu import CPU, ExecutionLimitExceeded
from rvsim.memory import DATA_SIZE, Memory, SimError, read_binary_file, write_data_out

EXIT_OK = 0
EXIT_IO_ERROR = 1
EXIT_LIMIT_EXCEEDED = 2
EXIT_RUNTIME_ERROR = 3

DATA_OUT_PATH = "data.out"  # 상대 경로 -> 현재 작업 디렉터리에 생성


def format_termination_line(pc, word):
    """종료 출력 한 줄. 두 숫자 모두 0x 접두사 + 선행 0 포함 소문자 8자리 16진수."""
    return "PC 0x%08x invalid instruction 0x%08x₩n" % (pc, word)


def error(message):
    sys.stderr.write(message + "₩n")


def main(argv):
    if len(argv) != 3:
        error("usage: python sim.py <text.bin> <data.bin>")
        return EXIT_IO_ERROR
    text_path, data_path = argv[1], argv[2]

    try:
        text = read_binary_file(text_path)
        data = read_binary_file(data_path)
    except OSError as e:
        error("error: cannot read input file: %s" % e)
        return EXIT_IO_ERROR

    # 명세의 유효 입력 범위를 벗어난 크기는 경고만 하고 계속 진행한다.
    if len(text) == 0 or len(text) % 4 != 0:
        error("warning: text.bin size %d is not a positive multiple of 4" % len(text))
    if len(data) > DATA_SIZE:
        error("warning: data.bin size %d exceeds %d bytes; extra bytes are ignored"
              % (len(data), DATA_SIZE))

    memory = Memory(text, data)
    cpu = CPU(memory)
    try:
        pc, word = cpu.run()
    except ExecutionLimitExceeded as e:
        error("error: %s (executed %d instructions)" % (e, cpu.executed))
        return EXIT_LIMIT_EXCEEDED
    except SimError as e:
        error("error: %s (executed %d instructions)" % (e, cpu.executed))
        return EXIT_RUNTIME_ERROR

    # 종료 처리: data.out 저장 -> 표준 출력 한 줄
    try:
        write_data_out(DATA_OUT_PATH, memory)
    except OSError as e:
        error("error: cannot write %s: %s" % (DATA_OUT_PATH, e))
        return EXIT_IO_ERROR

    # 텍스트 모드 stdout 은 Windows 에서 "₩n" 을 "₩r₩n" 으로 바꾸므로 바이트로 직접 쓴다.
```

```
75      sys.stdout.buffer.write(format_termination_line(pc, word).encode("ascii"))
76      sys.stdout.buffer.flush()
77      return EXIT_OK
78
79
80  if __name__ == "__main__":
81      sys.exit(main(sys.argv))
```

rvsim/__init__.py (7줄)

```
"""RV32I 부분집합(14개 명령어) 기능 시뮬레이터 패키지.

  bits.py    : 32비트 자르기, 부호 해석, 부호 확장, 비트 필드 추출
  memory.py  : text/data segment, little-endian 읽기·쓰기, 바이너리 파일 입출력
  decoder.py : 명령어 해독, 즉시값 생성, invalid instruction 판별
  cpu.py     : 레지스터·PC, fetch -> decode -> execute 루프, 명령어 실행
"""
```

rvsim/bits.py (53줄)

```
"""32비트 정수 · 부호 처리 도우미.

Python 의 int 는 크기 제한이 없고(임의 정밀도) 음수를 그대로 표현한다.
하드웨어의 32비트 레지스터처럼 동작하게 하려면 결과를 하위 32비트로 잘라야 한다.

이 시뮬레이터의 값 표현 규칙
  - 레지스터 · 메모리에 저장되는 값은 항상 0 ~ 0xFFFFFFFF 범위의
    '부호 없는 32비트 비트 패턴'으로 보관한다.
  - 즉시값(immediate)은 해독 단계에서 부호 확장하여 Python 의 음수/양수 정수로 둔다.
  - 부호 있는 해석이 필요한 곳(blt, bge 비교)에서만 to_signed32() 를 사용한다.
"""

MASK32 = 0xFFFFFFFF


def to_u32(value):
    """임의의 Python 정수를 하위 32비트 비트 패턴(0 ~ 0xFFFFFFFF)으로 자른다.

    예) to_u32(0xFFFFFFFF + 1) -> 0x00000000   (덧셈 결과의 자리올림 버림)
        to_u32(0 - 1)          -> 0xFFFFFFFF   (음수 -> 2의 보수 비트 패턴)
    """
    return value & MASK32


def to_signed32(value):
    """32비트 비트 패턴을 2의 보수 부호 있는 정수(-2^31 ~ 2^31-1)로 해석한다.

    예) 0xFFFFFFFF -> -1, 0x80000000 -> -2147483648, 0x7FFFFFFF -> 2147483647
    """
    value &= MASK32
    if value & 0x80000000:          # bit 31(부호 비트)이 1 이면 음수
        return value - 0x100000000  # 2^32 를 빼서 음수 값으로 바꾼다
    return value


def sign_extend(value, bit_count):
    """bit_count 비트 폭의 값을 부호 확장한다(최상위 비트가 1 이면 음수).

    예) sign_extend(0xFFF, 12) -> -1      sign_extend(0x800, 12) -> -2048
        sign_extend(0x7FF, 12) -> 2047    sign_extend(0x1000, 13) -> -4096
    """
    value &= (1 << bit_count) - 1
    if value & (1 << (bit_count - 1)):
        return value - (1 << bit_count)
    return value


def get_bits(value, hi, lo):
    """value 의 비트 [hi:lo] (양 끝 포함) 필드를 꺼낸다.

    예) get_bits(0x00700113, 6, 0) -> 0x13 (opcode)
    """
    return (value >> lo) & ((1 << (hi - lo + 1)) - 1)
```

rvsim/memory.py (88줄)

```python
"""메모리 모델과 바이너리 입출력.

주소 공간 (명세 2절, 6절)
  text segment : 0x00040000 부터 text.bin 크기만큼. 명령어 fetch 전용, 쓰기 불가.
  data segment : 0x10000000 ~ 0x100003FF (1,024 바이트). lw/sw 전용.

바이트 순서: little-endian (명령어·데이터 모두)
  주소 a 부터 네 바이트가 b0 b1 b2 b3 이면
  32비트 값 = b0 + (b1 << 8) + (b2 << 16) + (b3 << 24)
  예) 파일 바이트 78 56 34 12  ->  값 0x12345678
"""

TEXT_BASE = 0x00040000
DATA_BASE = 0x10000000
DATA_SIZE = 1024  # 0x400 바이트: 0x10000000 ~ 0x100003FF


class SimError(Exception):
    """명세의 유효 범위를 벗어난 동작(주소 범위·정렬 위반 등)."""


def read_word_le(buf, offset):
    """buf[offset] 부터 4 바이트를 little-endian 32비트 값으로 읽는다."""
    b0 = buf[offset]
    b1 = buf[offset + 1]
    b2 = buf[offset + 2]
    b3 = buf[offset + 3]
    return b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)


def write_word_le(buf, offset, value):
    """32비트 값을 little-endian 순서(하위 바이트 먼저)로 buf[offset] 부터 4 바이트에 쓴다."""
    buf[offset] = value & 0xFF
    buf[offset + 1] = (value >> 8) & 0xFF
    buf[offset + 2] = (value >> 16) & 0xFF
    buf[offset + 3] = (value >> 24) & 0xFF


class Memory:
    """text segment(읽기 전용)와 data segment(1,024 바이트)를 분리해 관리한다."""

    def __init__(self, text_bytes, data_bytes):
        # text: 수정할 수 없는 bytes 로 보관 -> text segment 쓰기 불가
        self.text = bytes(text_bytes)
        # data: 먼저 1,024 바이트 전체를 0 으로 초기화한 뒤 data.bin 내용을 앞에서부터 복사
        self.data = bytearray(DATA_SIZE)
        n = min(len(data_bytes), DATA_SIZE)
        self.data[0:n] = data_bytes[0:n]

    def fetch(self, addr):
        """명령어 인출: text segment 의 addr 에서 32비트 명령어 값을 읽는다."""
        offset = addr - TEXT_BASE
        if addr % 4 != 0 or offset < 0 or offset + 4 > len(self.text):
            raise SimError("instruction fetch outside text segment or misaligned: "
                           "pc=0x%08x" % addr)
        return read_word_le(self.text, offset)

    def _data_offset(self, addr):
        """data 주소를 self.data 배열의 인덱스로 바꾼다. 4바이트 정렬·범위를 검사한다."""
        offset = addr - DATA_BASE
        if addr % 4 != 0 or offset < 0 or offset + 4 > DATA_SIZE:
            raise SimError("data access outside data segment or misaligned: "
                           "addr=0x%08x" % addr)
        return offset

    def load_word(self, addr):
        """lw: data segment 에서 32비트 값을 little-endian 으로 읽는다."""
        return read_word_le(self.data, self._data_offset(addr))

    def store_word(self, addr, value):
        """sw: data segment 에 32비트 값을 little-endian 으로 쓴다."""
        write_word_le(self.data, self._data_offset(addr), value)


```

```
75 def read_binary_file(path):
76     """입력 파일을 바이너리 모드로 읽는다(ASCII/16진수 텍스트로 해석하지 않음)."""
77     with open(path, "rb") as f:
78         return f.read()
79 
80 
81 def write_data_out(path, memory):
82     """data segment 0x10000000 ~ 0x100003FF 의 1,024 바이트를 주소 증가 순서로 저장한다.
83 
84     "wb" 모드: 기존 파일을 덮어쓰고, 줄바꿈 변환 없이 바이트를 그대로 기록한다.
85     (텍스트 모드로 쓰면 Windows 에서 0x0A 바이트가 0x0D 0x0A 로 바뀌어 크기가 달라진다.)
86     """
87     with open(path, "wb") as f:
88         f.write(bytes(memory.data[0:DATA_SIZE]))
```

rvsim/decoder.py (148줄)

```
"""명령어 해독(decode)과 invalid instruction 판별.

RV32I 32비트 명령어 형식 (비트 번호 31 ~ 0)

        31       25 24   20 19   15 14  12 11         7 6      0
  R형  | funct7    |  rs2  |  rs1  |funct3|     rd     | opcode |
  I형  |     imm[11:0]     |  rs1  |funct3|     rd     | opcode |
  S형  | imm[11:5] |  rs2  |  rs1  |funct3|  imm[4:0]  | opcode |
  B형  |imm[12|10:5]| rs2  |  rs1  |funct3| imm[4:1|11]| opcode |
  U형  |            imm[31:12]            |     rd     | opcode |

지원 명령어 14개와 판별 조건 (이 표에 없는 조합은 모두 invalid instruction)

   명령어 형식  opcode    funct3  funct7
   add    R    0110011   000     0000000
   sub    R    0110011   000     0100000
   and    R    0110011   111     0000000
   or     R    0110011   110     0000000
   addi   I    0010011   000     -
   andi   I    0010011   111     -
   ori    I    0010011   110     -
   lw     I    0000011   010     -
   sw     S    0100011   010     -
   beq    B    1100011   000     -
   bne    B    1100011   001     -
   blt    B    1100011   100     -
   bge    B    1100011   101     -
   lui    U    0110111   -       -
"""

from .bits import get_bits, sign_extend

# opcode (명령어 비트 [6:0])
OPCODE_OP = 0b0110011      # 0x33  R형 연산: add, sub, and, or
OPCODE_OP_IMM = 0b0010011  # 0x13  I형 연산: addi, andi, ori
OPCODE_LOAD = 0b0000011    # 0x03  lw
OPCODE_STORE = 0b0100011   # 0x23  sw
OPCODE_BRANCH = 0b1100011  # 0x63  beq, bne, blt, bge
OPCODE_LUI = 0b0110111     # 0x37  lui

# opcode 별로 funct3(와 funct7) 조합 -> 명령어 이름
R_TYPE_TABLE = {           # (funct3, funct7) -> 이름
    (0b000, 0b0000000): "add",
    (0b000, 0b0100000): "sub",
    (0b111, 0b0000000): "and",
    (0b110, 0b0000000): "or",
}
OP_IMM_TABLE = {0b000: "addi", 0b111: "andi", 0b110: "ori"}
LOAD_TABLE = {0b010: "lw"}
STORE_TABLE = {0b010: "sw"}
BRANCH_TABLE = {0b000: "beq", 0b001: "bne", 0b100: "blt", 0b101: "bge"}


class Instruction:
    """해독된 명령어. 사용하지 않는 필드는 0 이다. imm 은 부호 확장된 Python 정수."""

    __slots__ = ("name", "rd", "rs1", "rs2", "imm")

    def __init__(self, name, rd=0, rs1=0, rs2=0, imm=0):
        self.name = name
        self.rd = rd
        self.rs1 = rs1
        self.rs2 = rs2
        self.imm = imm

    def __repr__(self):
        return "Instruction(%s, rd=%d, rs1=%d, rs2=%d, imm=%d)" % (
            self.name, self.rd, self.rs1, self.rs2, self.imm)


def imm_i(word):
    """I형 즉시값: imm[11:0] = word[31:20], 12비트 부호 확장."""
    return sign_extend(get_bits(word, 31, 20), 12)

```

```
75
76  def imm_s(word):
77      """S형 즉시값: imm[11:5] = word[31:25], imm[4:0] = word[11:7], 12비트 부호 확장."""
78      imm = (get_bits(word, 31, 25) << 5) | get_bits(word, 11, 7)
79      return sign_extend(imm, 12)
80
81
82  def imm_b(word):
83      """B형 즉시값(분기 오프셋, 바이트 단위). 13비트 부호 확장, imm[0] 은 항상 0.
84
85      imm[12]   = word[31]
86      imm[11]   = word[7]
87      imm[10:5] = word[30:25]
88      imm[4:1]  = word[11:8]
89      """
90      imm = ((get_bits(word, 31, 31) << 12)
91             | (get_bits(word, 7, 7) << 11)
92             | (get_bits(word, 30, 25) << 5)
93             | (get_bits(word, 11, 8) << 1))
94      return sign_extend(imm, 13)
95
96
97  def imm_u(word):
98      """U형 즉시값: word[31:12] 를 상위 20비트에 그대로 두고 하위 12비트는 0."""
99      return word & 0xFFFFF000
100
101
102 def decode(word):
103     """32비트 명령어 값을 해독한다.
104
105     지원하는 14개 명령어이면 Instruction 을, 아니면 None(invalid instruction)을 반환한다.
106     opcode 뿐 아니라 funct3, (R형은) funct7 까지 모두 일치해야 지원 명령어로 인정한다.
107     """
108     opcode = get_bits(word, 6, 0)
109     rd = get_bits(word, 11, 7)
110     funct3 = get_bits(word, 14, 12)
111     rs1 = get_bits(word, 19, 15)
112     rs2 = get_bits(word, 24, 20)
113     funct7 = get_bits(word, 31, 25)
114
115     if opcode == OPCODE_OP:                                 # R형
116         name = R_TYPE_TABLE.get((funct3, funct7))
117         if name is None:
118             return None
119         return Instruction(name, rd=rd, rs1=rs1, rs2=rs2)
120
121     if opcode == OPCODE_OP_IMM:                             # I형 연산
122         name = OP_IMM_TABLE.get(funct3)
123         if name is None:
124             return None
125         return Instruction(name, rd=rd, rs1=rs1, imm=imm_i(word))
126
127     if opcode == OPCODE_LOAD:                               # I형 lw
128         name = LOAD_TABLE.get(funct3)
129         if name is None:
130             return None
131         return Instruction(name, rd=rd, rs1=rs1, imm=imm_i(word))
132
133     if opcode == OPCODE_STORE:                              # S형 sw
134         name = STORE_TABLE.get(funct3)
135         if name is None:
136             return None
137         return Instruction(name, rs1=rs1, rs2=rs2, imm=imm_s(word))
138
139     if opcode == OPCODE_BRANCH:                             # B형 분기
140         name = BRANCH_TABLE.get(funct3)
141         if name is None:
142             return None
143         return Instruction(name, rs1=rs1, rs2=rs2, imm=imm_b(word))
144
145     if opcode == OPCODE_LUI:                                # U형 lui
146         return Instruction("lui", rd=rd, imm=imm_u(word))
147
148     return None                                             # 지원하지 않는 opcode
```

rvsim/cpu.py (104줄)

```
"""CPU 상태(PC, 레지스터 x0~x31)와 fetch -> decode -> execute 실행 루프."""

from .bits import to_signed32, to_u32
from .decoder import decode
from .memory import TEXT_BASE

MAX_INSTRUCTIONS = 1000000  # 정상 명령어 최대 실행 개수 (명세 6절)


class ExecutionLimitExceeded(Exception):
    """정상 명령어를 한도만큼 실행한 뒤 다음 fetch 도 지원 명령어인 경우(비정상 종료)."""

    def __init__(self, pc, word):
        super().__init__("execution limit exceeded: pc=0x%08x instruction=0x%08x"
                         % (pc, word))
        self.pc = pc
        self.word = word


class CPU:
    def __init__(self, memory):
        self.memory = memory
        self.regs = [0] * 32   # x0 ~ x31 모두 0 으로 시작 (스택 포인터 초기화 없음)
        self.pc = TEXT_BASE    # 초기 PC = 0x00040000
        self.executed = 0      # 지금까지 실행을 마친 정상 명령어 수

    def write_reg(self, index, value):
        """레지스터 쓰기. x0 는 항상 0 이므로 쓰기를 무시하고, 값은 하위 32비트만 저장한다."""
        if index != 0:
            self.regs[index] = to_u32(value)

    def run(self, limit=MAX_INSTRUCTIONS):
        """invalid instruction 을 fetch 할 때까지 실행한다.

        반환값: (invalid instruction 을 fetch 한 주소, 그 32비트 명령어 값)
        invalid instruction 자체는 실행하지 않는다.
        """
        while True:
            fetch_pc = self.pc                      # fetch 당시 주소를 따로 보존
            word = self.memory.fetch(fetch_pc)      # 1) fetch
            inst = decode(word)                     # 2) decode
            if inst is None:                        # 지원하지 않는 명령어 -> 실행 없이 종료
                return fetch_pc, word
            if self.executed >= limit:              # 이미 한도만큼 실행했는데 또 정상 명령어
                raise ExecutionLimitExceeded(fetch_pc, word)
            self.pc = self.execute(inst, fetch_pc)  # 3) execute + PC 갱신
            self.executed += 1

    def execute(self, inst, pc):
        """명령어 하나를 실행하고 다음 PC 를 반환한다. pc 는 이 명령어의 주소."""
        name = inst.name
        rs1 = self.regs[inst.rs1]   # x0 은 쓰기가 무시되므로 항상 0 이 읽힌다
        rs2 = self.regs[inst.rs2]
        imm = inst.imm              # 부호 확장된 즉시값 (Python 정수, 음수 가능)
        next_pc = pc + 4            # 분기하지 않으면 다음 명령어

        # ---- R형 산술·논리 ----
        if name == "add":
            self.write_reg(inst.rd, rs1 + rs2)         # write_reg 에서 하위 32비트만 저장
        elif name == "sub":
            self.write_reg(inst.rd, rs1 - rs2)         # 음수 결과도 2의 보수 비트 패턴으로
        elif name == "and":
            self.write_reg(inst.rd, rs1 & rs2)
        elif name == "or":
            self.write_reg(inst.rd, rs1 | rs2)

        # ---- I형 산술·논리 (즉시값은 부호 확장된 값) ----
        elif name == "addi":
            self.write_reg(inst.rd, rs1 + imm)
        elif name == "andi":
            self.write_reg(inst.rd, rs1 & to_u32(imm))  # 예) imm=-1 -> 0xFFFFFFFF
        elif name == "ori":
            self.write_reg(inst.rd, rs1 | to_u32(imm))

```

```
        # ---- U형 ----
        elif name == "lui":
            self.write_reg(inst.rd, imm)                # imm = 상위 20비트 << 12

        # ---- 메모리 (유효 주소 = rs1 + 부호 확장 오프셋, 32비트로 자름) ----
        elif name == "lw":
            addr = to_u32(rs1 + imm)
            self.write_reg(inst.rd, self.memory.load_word(addr))
        elif name == "sw":
            addr = to_u32(rs1 + imm)
            self.memory.store_word(addr, rs2)

        # ---- B형 분기 (분기 목표 = 분기 명령어 자신의 주소 + 오프셋) ----
        elif name == "beq":
            if rs1 == rs2:
                next_pc = pc + imm
        elif name == "bne":
            if rs1 != rs2:
                next_pc = pc + imm
        elif name == "blt":                             # 부호 있는 비교
            if to_signed32(rs1) < to_signed32(rs2):
                next_pc = pc + imm
        elif name == "bge":                             # 부호 있는 비교
            if to_signed32(rs1) >= to_signed32(rs2):
                next_pc = pc + imm

        else:
            raise AssertionError("decode() returned unknown instruction: " + name)

        return to_u32(next_pc)
```

tests/asm.py (214줄)

```
"""테스트 입력 생성용 미니 어셈블러(인코더).

시뮬레이터(rvsim)의 해독 코드와 독립적으로 작성했다. rvsim 을 import 하지 않는다.
(해독기와 인코더가 같은 코드를 공유하면 같은 실수가 양쪽에 숨어 검증이 되지 않기 때문)

지원 문법
  label:                      레이블 정의 (분기 목표)
  add  rd, rs1, rs2           sub / and / or 동일
  addi rd, rs1, imm           andi / ori 동일
  lui  rd, imm20              imm20 은 0 ~ 0xFFFFF (상위 20비트)
  lw   rd, imm(rs1)
  sw   rs2, imm(rs1)
  beq  rs1, rs2, label|offset  bne / blt / bge 동일 (offset 은 바이트 단위)
  nop                         addi x0, x0, 0
  .word value                 32비트 값을 그대로 배치 (invalid instruction 등)
  # 이후는 주석

invalid instruction 테스트용으로, 지원하지 않는 실제 RV32I/RV32M 명령어도 인코딩할 수 있다.
  R형: xor sll srl sra slt sltu mul       I형: slti sltiu xori slli srli srai
  load: lb lh lbu lhu   store: sb sh       branch: bltu bgeu
  jal rd, offset   jalr rd, imm(rs1)   auipc rd, imm20   ecall   ebreak
"""

import re
import struct

TEXT_BASE = 0x00040000

REGS = {"x%d" % i: i for i in range(32)}
REGS["zero"] = 0

# 지원 명령어 14개
R_OPS = {"add": (0b000, 0b0000000), "sub": (0b000, 0b0100000),
         "and": (0b111, 0b0000000), "or": (0b110, 0b0000000)}
I_OPS = {"addi": 0b000, "andi": 0b111, "ori": 0b110}
BRANCH_OPS = {"beq": 0b000, "bne": 0b001, "blt": 0b100, "bge": 0b101}

# invalid instruction 테스트용(시뮬레이터는 지원하지 않음)
R_OPS_UNSUPPORTED = {"sll": (0b001, 0), "slt": (0b010, 0), "sltu": (0b011, 0),
                     "xor": (0b100, 0), "srl": (0b101, 0), "sra": (0b101, 0b0100000),
                     "mul": (0b000, 0b0000001)}
I_OPS_UNSUPPORTED = {"slti": 0b010, "sltiu": 0b011, "xori": 0b100}
SHIFT_OPS_UNSUPPORTED = {"slli": (0b001, 0), "srli": (0b101, 0), "srai": (0b101, 0b0100000)}
LOAD_OPS_UNSUPPORTED = {"lb": 0b000, "lh": 0b001, "lbu": 0b100, "lhu": 0b101}
STORE_OPS_UNSUPPORTED = {"sb": 0b000, "sh": 0b001}
BRANCH_OPS_UNSUPPORTED = {"bltu": 0b110, "bgeu": 0b111}

OP, OP_IMM, LOAD, STORE, BRANCH, LUI = 0x33, 0x13, 0x03, 0x23, 0x63, 0x37
AUIPC, JAL, JALR, SYSTEM = 0x17, 0x6F, 0x67, 0x73


class AsmError(Exception):
    pass


def _check_range(value, lo, hi, what):
    if not lo <= value <= hi:
        raise AsmError("%s %d out of range [%d, %d]" % (what, value, lo, hi))


def enc_r(funct7, rs2, rs1, funct3, rd, opcode):
    return (funct7 << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode


def enc_i(imm, rs1, funct3, rd, opcode):
    _check_range(imm, -2048, 2047, "I-type imm")
    return ((imm & 0xFFF) << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode


def enc_s(imm, rs2, rs1, funct3, opcode):
    _check_range(imm, -2048, 2047, "S-type imm")
    imm &= 0xFFF
    return (((imm >> 5) & 0x7F) << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) W
        | ((imm & 0x1F) << 7) | opcode
```

```
75
76
77  def enc_b(offset, rs2, rs1, funct3, opcode):
78      _check_range(offset, -4096, 4094, "B-type offset")
79      if offset % 2:
80          raise AsmError("B-type offset must be even: %d" % offset)
81      imm = offset & 0x1FFF
82      return (((imm >> 12) & 1) << 31) | (((imm >> 5) & 0x3F) << 25) | (rs2 << 20) ₩
83          | (rs1 << 15) | (funct3 << 12) | (((imm >> 1) & 0xF) << 8) ₩
84          | (((imm >> 11) & 1) << 7) | opcode
85
86
87  def enc_u(imm20, rd, opcode):
88      _check_range(imm20, 0, 0xFFFFF, "U-type imm20")
89      return (imm20 << 12) | (rd << 7) | opcode
90
91
92  def enc_j(offset, rd, opcode):
93      _check_range(offset, -(1 << 20), (1 << 20) - 2, "J-type offset")
94      imm = offset & 0x1FFFFF
95      return (((imm >> 20) & 1) << 31) | (((imm >> 1) & 0x3FF) << 21) ₩
96          | (((imm >> 11) & 1) << 20) | (((imm >> 12) & 0xFF) << 12) | (rd << 7) | opcode
97
98
99  def _reg(tok):
100     tok = tok.strip()
101     if tok not in REGS:
102         raise AsmError("bad register: %r" % tok)
103     return REGS[tok]
104
105
106 def _int(tok):
107     return int(tok.strip(), 0)
108
109
110 _MEM_RE = re.compile(r"^Ws*(-?(?:0x[0-9a-fA-F]+|Wd+))Ws*W(Ws*(Ww+)Ws*W)Ws*$")
111
112
113 def _mem(tok):
114     m = _MEM_RE.match(tok)
115     if not m:
116         raise AsmError("bad memory operand: %r" % tok)
117     return int(m.group(1), 0), _reg(m.group(2))
118
119
120 def _parse(source):
121     """소스를 (주소, 명령어 이름, 피연산자 목록, 원본 줄) 목록과 레이블 표로 바꾼다."""
122     items, labels = [], {}
123     addr = TEXT_BASE
124     for raw in source.splitlines():
125         line = raw.split("#", 1)[0].strip()
126         while line:
127             m = re.match(r"^(Ww+)Ws*:Ws*(.*)$", line)
128             if not m:
129                 break
130             if m.group(1) in labels:
131                 raise AsmError("duplicate label: " + m.group(1))
132             labels[m.group(1)] = addr
133             line = m.group(2).strip()
134         if not line:
135             continue
136         parts = line.split(None, 1)
137         mnemonic = parts[0].lower()
138         operands = [p.strip() for p in parts[1].split(",")] if len(parts) > 1 else []
139         items.append((addr, mnemonic, operands, raw.strip()))
140         addr += 4
141     return items, labels
142
143
144 def _target(tok, labels, pc):
145     tok = tok.strip()
146     if tok in labels:
147         return labels[tok] - pc
148     return _int(tok)
149
150
```

```
def _encode(mn, ops, labels, pc):
    if mn in R_OPS or mn in R_OPS_UNSUPPORTED:
        funct3, funct7 = R_OPS.get(mn) or R_OPS_UNSUPPORTED[mn]
        return enc_r(funct7, _reg(ops[2]), _reg(ops[1]), funct3, _reg(ops[0]), OP)
    if mn in I_OPS or mn in I_OPS_UNSUPPORTED:
        funct3 = I_OPS[mn] if mn in I_OPS else I_OPS_UNSUPPORTED[mn]
        return enc_i(_int(ops[2]), _reg(ops[1]), funct3, _reg(ops[0]), OP_IMM)
    if mn in SHIFT_OPS_UNSUPPORTED:
        funct3, funct7 = SHIFT_OPS_UNSUPPORTED[mn]
        shamt = _int(ops[2])
        _check_range(shamt, 0, 31, "shamt")
        return enc_r(funct7, shamt, _reg(ops[1]), funct3, _reg(ops[0]), OP_IMM)
    if mn == "nop":
        return enc_i(0, 0, 0b000, 0, OP_IMM)
    if mn == "lui" or mn == "auipc":
        return enc_u(_int(ops[1]), _reg(ops[0]), LUI if mn == "lui" else AUIPC)
    if mn == "lw" or mn in LOAD_OPS_UNSUPPORTED:
        imm, rs1 = _mem(ops[1])
        funct3 = 0b010 if mn == "lw" else LOAD_OPS_UNSUPPORTED[mn]
        return enc_i(imm, rs1, funct3, _reg(ops[0]), LOAD)
    if mn == "sw" or mn in STORE_OPS_UNSUPPORTED:
        imm, rs1 = _mem(ops[1])
        funct3 = 0b010 if mn == "sw" else STORE_OPS_UNSUPPORTED[mn]
        return enc_s(imm, _reg(ops[0]), rs1, funct3, STORE)
    if mn in BRANCH_OPS or mn in BRANCH_OPS_UNSUPPORTED:
        funct3 = BRANCH_OPS[mn] if mn in BRANCH_OPS else BRANCH_OPS_UNSUPPORTED[mn]
        return enc_b(_target(ops[2], labels, pc), _reg(ops[1]), _reg(ops[0]), funct3, BRANCH)
    if mn == "jal":
        return enc_j(_target(ops[1], labels, pc), _reg(ops[0]), JAL)
    if mn == "jalr":
        imm, rs1 = _mem(ops[1])
        return enc_i(imm, rs1, 0b000, _reg(ops[0]), JALR)
    if mn == "ecall":
        return SYSTEM
    if mn == "ebreak":
        return (1 << 20) | SYSTEM
    if mn == ".word":
        value = _int(ops[0])
        _check_range(value, 0, 0xFFFFFFFF, ".word value")
        return value
    raise AsmError("unknown mnemonic: " + mn)


def assemble(source):
    """어셈블리 소스 -> 32비트 명령어 값 목록 (주소 0x00040000 부터 4바이트 간격)."""
    items, labels = _parse(source)
    words = []
    for pc, mn, ops, raw in items:
        try:
            words.append(_encode(mn, ops, labels, pc))
        except (AsmError, IndexError, ValueError) as e:
            raise AsmError("0x%08x: %s  <- %s" % (pc, e, raw))
    return words


def listing(source):
    """(주소, 명령어 값, 원본 줄) 목록. 테스트 문서화용."""
    items, _ = _parse(source)
    return [(pc, w, raw) for (pc, _, _, raw), w in zip(items, assemble(source))]


def words_to_bytes(words):
    """32비트 값 목록 -> little-endian 바이트열 (struct '<I' 사용)."""
    return b"".join(struct.pack("<I", w) for w in words)
```

tests/cases.py (641줄)

```
"""자체 테스트 케이스 정의: 프로그램(어셈블리), data.bin, 손으로 도출한 기대 결과.

기대 결과는 시뮬레이터 실행 결과를 복사한 것이 아니라, 각 명령어의 의미에 따라
직접 계산한 값이다. 계산 근거를 각 줄의 주석에 적었다.

필드
  name          : 케이스 이름 (tests/cases/<name>/ 디렉터리)
  purpose       : 테스트 목적
  source        : 어셈블리 소스 (tests/asm.py 문법). 0x00040000 부터 배치
  data          : data.bin 내용 (bytes)
  stdout        : 기대 표준 출력 한 줄 (비정상 종료면 "")
  expect_words  : {data 오프셋: 32비트 값} - 실행 후 바뀌는 워드 (나머지는 data.bin 그대로)
  expect_bytes  : {data 오프셋: "16진 바이트열"} - 바이트 단위로 확인할 위치
  exit_code     : 기대 종료 코드 (0 정상, 2 실행 한도 초과, 3 범위·정렬 위반)
  check_text    : (선택) text.bin 이 이 바이트열과 같아야 함 (명세 예시 대조)
"""

import struct

DATA_SIZE = 1024


def words(*values):
    """32비트 값들을 little-endian data.bin 바이트열로 만든다."""
    return b"".join(struct.pack("<I", v & 0xFFFFFFFF) for v in values)


def line(pc, word):
    return "PC 0x%08x invalid instruction 0x%08x₩n" % (pc, word)


CASES = []


def case(name, purpose, source, data=b"", stdout="", expect_words=None,
         expect_bytes=None, exit_code=0, check_text=None):
    CASES.append(dict(name=name, purpose=purpose, source=source, data=data,
                      stdout=stdout, expect_words=expect_words or {},
                      expect_bytes=expect_bytes or {}, exit_code=exit_code,
                      check_text=check_text))


# ---------------------------------------------------------------------------
# 1. 명세 예시와 기본 입출력
# ---------------------------------------------------------------------------
case("01_spec_example",
     "명세 7절 예시를 그대로 재현한다. text.bin 바이트열까지 명세와 대조한다.",
     """
     lui  x1, 0x10000      # x1 = 0x10000 << 12 = 0x10000000
     addi x2, x0, 7        # x2 = 7
     addi x3, x0, 5        # x3 = 5
     add  x4, x2, x3       # x4 = 12 = 0x0000000c
     sw   x4, 0(x1)        # [0x10000000] = 0x0000000c -> 바이트 0c 00 00 00
     .word 0x00000000      # 0x00040014: invalid -> 종료
     """,
     data=bytes(DATA_SIZE),
     stdout=line(0x00040014, 0x00000000),
     expect_bytes={0: "0c000000"},
     check_text=bytes.fromhex("b7000010 13017000 93015000 33023100 23a04000 00000000"))

case("02_immediate_invalid",
     "첫 명령어가 invalid. 아무것도 실행하지 않고 PC 0x00040000 을 출력하며, "
     "data.out 은 data.bin 1,024 바이트와 완전히 같아야 한다(0x0a, 0x0d, 0x1a 바이트 포함).",
     """
     .word 0x00000000      # 0x00040000: 첫 fetch 부터 invalid
     """,
     data=bytes((i * 7 + 3) & 0xFF for i in range(DATA_SIZE)),
     stdout=line(0x00040000, 0x00000000))

# ---------------------------------------------------------------------------
# 2. 레지스터·산술·논리·즉시값
# ---------------------------------------------------------------------------
case("03_x0_hardwired",
     "x0 에 대한 쓰기(addi, lui, add, sub, lw)가 모두 무시되고 x0 를 읽으면 항상 0 인지 확인.",
```

```
    """
    lui  x1, 0x10000      # x1 = 0x10000000
    addi x0, x0, 5        # 무시
    lui  x0, 0xfffff      # 무시
    addi x2, x0, 7        # x2 = 0 + 7 = 7
    add  x0, x2, x2       # 무시
    sub  x0, x0, x2       # 무시
    lw   x0, 0(x1)        # 메모리 값 0x11223344 를 읽지만 무시
    sw   x0, 4(x1)        # [+4] = 0 (0xdeadbeef 를 덮어씀)
    add  x3, x0, x2       # x3 = 0 + 7 = 7
    sw   x3, 8(x1)        # [+8] = 7 (0xffffffff 를 덮어씀)
    .word 0               # 0x00040028
    """,
    data=words(0x11223344, 0xDEADBEEF, 0xFFFFFFFF),
    stdout=line(0x00040028, 0),
    expect_words={4: 0x00000000, 8: 0x00000007})

case("04_add_sub_wrap",
    "add/sub 결과를 하위 32비트로 유지(오버플로 무시, 음수는 2의 보수)하는지 확인.",
    """
    lui  x1, 0x10000      # x1 = 0x10000000
    lui  x2, 0x80000      # x2 = 0x80000000
    addi x2, x2, -1       # x2 = 0x7fffffff (INT_MAX)
    addi x3, x0, 1        # x3 = 1
    add  x4, x2, x3       # 0x7fffffff + 1 = 0x80000000 (부호 오버플로, 예외 없음)
    sw   x4, 0(x1)
    sub  x5, x0, x3       # 0 - 1 = -1 = 0xffffffff
    sw   x5, 4(x1)
    add  x6, x4, x4       # 0x80000000 * 2 = 0x1_00000000 -> 하위 32비트 0x00000000
    sw   x6, 8(x1)
    add  x7, x5, x5       # 0xffffffff * 2 = 0x1_fffffffe -> 0xfffffffe
    sw   x7, 12(x1)
    sub  x8, x4, x3       # 0x80000000 - 1 = 0x7fffffff
    sw   x8, 16(x1)
    sub  x9, x3, x4       # 1 - 0x80000000 = -2147483647 -> 0x80000001
    sw   x9, 20(x1)
    .word 0               # 0x00040040
    """,
    data=b"Wxaa" * 24,
    stdout=line(0x00040040, 0),
    expect_words={0: 0x80000000, 4: 0xFFFFFFFF, 8: 0x00000000,
                  12: 0xFFFFFFFE, 16: 0x7FFFFFFF, 20: 0x80000001})

case("05_addi_imm",
    "I형 12비트 즉시값의 부호 확장과 경계값(-2048, 2047), addi 자리올림 버림 확인.",
    """
    lui  x1, 0x10000
    addi x2, x0, -1       # imm 0xfff -> -1 -> x2 = 0xffffffff
    addi x3, x0, -2048    # imm 0x800 -> -2048 -> 0xfffff800
    addi x4, x0, 2047     # imm 0x7ff -> 0x000007ff
    addi x5, x4, 1        # 0x7ff + 1 = 0x00000800
    addi x6, x2, 1        # 0xffffffff + 1 = 0x1_00000000 -> 0x00000000
    addi x7, x3, -2048    # 0xfffff800 - 0x800 = 0xfffff000
    addi x8, x8, 0x123    # 초기 레지스터 0 -> 0x00000123
    sw   x2, 0(x1)
    sw   x3, 4(x1)
    sw   x4, 8(x1)
    sw   x5, 12(x1)
    sw   x6, 16(x1)
    sw   x7, 20(x1)
    sw   x8, 24(x1)
    .word 0               # 0x0004003c
    """,
    data=b"Wxaa" * 28,
    stdout=line(0x0004003C, 0),
    expect_words={0: 0xFFFFFFFF, 4: 0xFFFFF800, 8: 0x000007FF, 12: 0x00000800,
                  16: 0x00000000, 20: 0xFFFFF000, 24: 0x00000123})

case("06_logic",
    "and/or/andi/ori. andi/ori 의 즉시값도 부호 확장됨(-1 -> 0xffffffff, -2048 -> 0xfffff800).",
    """
    lui  x1, 0x10000
    lui  x2, 0x12345
    addi x2, x2, 0x678    # x2 = 0x12345678
    lui  x3, 0xff00f
    addi x3, x3, 0x0f0    # x3 = 0xff00f0f0
```

```
    and  x4, x2, x3       # 12&ff 34&00 56&f0 78&f0 = 0x12005070
    or   x5, x2, x3       # 12|ff 34|00 56|f0 78|f0 = 0xff34f6f8
    andi x6, x2, -1       # & 0xffffffff = 0x12345678 (그대로)
    andi x7, x2, 0x7ff    # & 0x000007ff = 0x00000678
    ori  x8, x2, -2048    # | 0xfffff800 = 0xfffffe78
    andi x9, x2, -256     # & 0xffffff00 = 0x12345600
    ori  x10, x0, 0x555   # 0 | 0x555 = 0x00000555
    sw   x4, 0(x1)
    sw   x5, 4(x1)
    sw   x6, 8(x1)
    sw   x7, 12(x1)
    sw   x8, 16(x1)
    sw   x9, 20(x1)
    sw   x10, 24(x1)
    .word 0               # 0x0004004c
    """,
    data=b"₩xaa" * 28,
    stdout=line(0x0004004C, 0),
    expect_words={0: 0x12005070, 4: 0xFF34F6F8, 8: 0x12345678, 12: 0x00000678,
                  16: 0xFFFFFE78, 20: 0x12345600, 24: 0x00000555})

case("07_lui",
    "lui 는 imm20<<12 를 쓰고 하위 12비트를 0 으로 만든다. lui+addi 로 32비트 상수 만들기"
    "(하위 12비트가 0x800 이상이면 상위를 +1 하고 음수 addi).",
    """
    lui  x1, 0x10000
    lui  x2, 0xfffff      # 0xfffff000
    lui  x3, 0x1          # 0x00001000
    lui  x4, 0x12346
    addi x4, x4, -1       # 0x12346000 - 1 = 0x12345fff
    lui  x5, 0x80000      # 0x80000000
    lui  x6, 0xdeadc
    addi x6, x6, -0x111   # 0xdeadc000 - 0x111 = 0xdeadbeef
    addi x7, x0, 0x7ff    # x7 = 0x000007ff
    lui  x7, 0x54321      # 하위 비트까지 덮어씀 -> 0x54321000 (0x543217ff 아님)
    sw   x2, 0(x1)
    sw   x3, 4(x1)
    sw   x4, 8(x1)
    sw   x5, 12(x1)
    sw   x6, 16(x1)
    sw   x7, 20(x1)
    .word 0               # 0x00040040
    """,
    data=b"₩xaa" * 24,
    stdout=line(0x00040040, 0),
    expect_words={0: 0xFFFFF000, 4: 0x00001000, 8: 0x12345FFF, 12: 0x80000000,
                  16: 0xDEADBEEF, 20: 0x54321000})

case("08_wrap_compare",
    "레지스터 값이 항상 32비트 비트 패턴인지 확인. addi 로 만든 -1 과 lw 로 읽은 0xffffffff 는 "
    "같은 값이어야 하고(beq 분기), 0xffffffff + 1 은 0 이어야 한다. "
    "(32비트로 자르지 않으면 sw 결과는 우연히 맞아도 beq/bne 비교가 틀어진다. 버그 시 x10 = 3)",
    """
    lui  x1, 0x10000
    lw   x2, 0(x1)        # 0xffffffff (메모리에서 읽음)
    addi x3, x0, -1       # 0 + (-1) -> 0xffffffff (연산 결과)
    addi x10, x0, 0
    beq  x2, x3, A        # 같은 비트 패턴 -> 분기 (bit0 안 켜짐)
    ori  x10, x10, 1
A:  addi x4, x2, 1        # 0xffffffff + 1 = 0x1_00000000 -> 0x00000000
    beq  x4, x0, B        # 0 == 0 -> 분기 (bit1 안 켜짐)
    ori  x10, x10, 2
B:  add  x5, x2, x2       # 0x1_fffffffe -> 0xfffffffe
    lui  x6, 0xfffff      # 0xfffff000
    ori  x6, x6, -2       # 0xfffff000 | 0xfffffffe = 0xfffffffe
    bne  x5, x6, C        # 같음 -> 진행 (bit2 켜짐)
    ori  x10, x10, 4
C:  sw   x10, 4(x1)       # x10 = 4
    .word 0               # 0x0004003c
    """,
    data=words(0xFFFFFFFF),
    stdout=line(0x0004003C, 0),
    expect_words={4: 0x00000004})

# ---------------------------------------------------------------------------
# 3. 메모리 · 바이트 순서
```

```
# ---------------------------------------------------------------------------
_endian_data = bytearray(DATA_SIZE)
_endian_data[0:4] = bytes.fromhex("78563412")
_endian_data[4:8] = bytes.fromhex("efbeadde")
_endian_data[8:12] = bytes.fromhex("00000080")
_endian_data[1020:1024] = bytes.fromhex("01020304")

case("09_lw_sw_endian",
     "little-endian lw/sw. 78 56 34 12 -> 0x12345678, 저장 시 하위 바이트부터. "
     "데이터 영역 첫 워드(0x10000000)와 마지막 워드(0x100003fc), 음수 오프셋.",
     """
     lui  x1, 0x10000      # x1 = 0x10000000
     lw   x2, 0(x1)        # 78 56 34 12 -> 0x12345678
     lw   x3, 4(x1)        # ef be ad de -> 0xdeadbeef
     lw   x4, 1020(x1)     # 01 02 03 04 -> 0x04030201 (마지막 워드)
     sw   x2, 16(x1)       # [+16] = 78 56 34 12
     addi x5, x1, 1024     # x5 = 0x10000400 (데이터 영역 끝 다음 주소)
     sw   x3, -4(x5)       # [0x100003fc] = ef be ad de
     sw   x4, -1024(x5)    # [0x10000000] = 01 02 03 04
     lw   x6, 8(x1)        # 00 00 00 80 -> 0x80000000
     addi x7, x6, -1       # 0x7fffffff
     sw   x7, 20(x1)       # [+20] = ff ff ff 7f
     .word 0               # 0x0004002c
     """,
     data=bytes(_endian_data),
     stdout=line(0x0004002C, 0),
     expect_bytes={0: "01020304", 4: "efbeadde", 8: "00000080", 16: "78563412",
                   20: "ffffff7f", 1020: "efbeadde"})

case("10_sw_lw_imm_split",
     "S형 즉시값은 imm[11:5]=bit31:25, imm[4:0]=bit11:7 로 나뉜다. 큰 양수(2044)·음수(-516, -1024) "
     "오프셋으로 sw/lw 주소 계산 확인.",
     """
     lui  x1, 0x10000      # 0x10000000
     addi x3, x1, -1024    # x3 = 0x0ffffc00 (데이터 영역 밖이지만 기준 주소로만 사용)
     addi x4, x1, 1024     # x4 = 0x10000400
     lui  x2, 0xa1b2c
     addi x2, x2, 0x3d4    # x2 = 0xa1b2c3d4
     sw   x2, 2044(x3)     # 0x0ffffc00 + 0x7fc = 0x100003fc
     sw   x2, -516(x4)     # 0x10000400 - 0x204 = 0x100001fc
     addi x5, x0, 0x11
     sw   x5, -1024(x4)    # 0x10000400 - 0x400 = 0x10000000
     lw   x6, -4(x4)       # [0x100003fc] = 0xa1b2c3d4
     addi x6, x6, 1        # 0xa1b2c3d5
     sw   x6, 2040(x3)     # 0x0ffffc00 + 0x7f8 = 0x100003f8
     lw   x7, 2044(x3)     # [0x100003fc] = 0xa1b2c3d4
     sw   x7, 4(x1)        # [0x10000004] = 0xa1b2c3d4
     .word 0               # 0x00040038
     """,
     stdout=line(0x00040038, 0),
     expect_words={0x000: 0x00000011, 0x004: 0xA1B2C3D4, 0x1FC: 0xA1B2C3D4,
                   0x3F8: 0xA1B2C3D5, 0x3FC: 0xA1B2C3D4})

case("11_data_empty",
     "data.bin 이 0 바이트여도 데이터 영역 1,024 바이트는 0 으로 초기화되고 data.out 은 1,024 바이트. "
     "0x0a/0x0d 바이트를 바이너리로 정확히 기록하는지(텍스트 모드 줄바꿈 변환 금지) 확인.",
     """
     lui  x1, 0x10000
     lw   x2, 512(x1)      # 초기화된 0
     addi x2, x2, 0x0d     # 0x0000000d
     sw   x2, 0(x1)        # 0d 00 00 00
     addi x3, x0, 0x0a
     sw   x3, 1020(x1)     # 0a 00 00 00 (마지막 워드)
     lui  x4, 0x0a0d1      # 하위 12비트 0xa0d >= 0x800 이므로 상위 +1
     addi x4, x4, -0x5f3   # 0x0a0d1000 - 0x5f3 = 0x0a0d0a0d
     sw   x4, 4(x1)        # 0d 0a 0d 0a
     .word 0               # 0x00040024
     """,
     data=b"",
     stdout=line(0x00040024, 0),
     expect_bytes={0: "0d000000", 4: "0d0a0d0a", 1020: "0a000000"})

case("12_data_partial",
     "data.bin 이 6 바이트(4의 배수 아님)일 때 나머지는 0. 01 02 03 04 05 06 00 00 ...",
     """
     lui  x1, 0x10000
```

```
303       lw   x2, 4(x1)        # 05 06 00 00 -> 0x00000605
304       sw   x2, 8(x1)
305       lw   x3, 0(x1)        # 01 02 03 04 -> 0x04030201
306       sw   x3, 12(x1)
307       .word 0               # 0x00040014
308       """,
309       data=bytes.fromhex("010203040506"),
310       stdout=line(0x00040014, 0),
311       expect_bytes={0: "0102030405060000", 8: "0506000001020304"})
312
313  # ------------------------------------------------------------------------
314  # 4. 분기 · PC 갱신
315  # ------------------------------------------------------------------------
316  case("13_beq_bne",
317       "beq/bne 의 분기/비분기. 분기하면 다음 ori 를 건너뛴다. x10 비트가 1 = 그 ori 가 실행됨.",
318       """
319       lui  x1, 0x10000
320       addi x2, x0, 5
321       addi x3, x0, 5
322       addi x4, x0, 7
323       addi x10, x0, 0
324       beq  x2, x3, L1       # 5 == 5 -> 분기 (bit0 안 켜짐)
325       ori  x10, x10, 1
326  L1:  beq  x2, x4, L2       # 5 == 7 거짓 -> 진행 (bit1 켜짐)
327       ori  x10, x10, 2
328  L2:  bne  x2, x4, L3       # 5 != 7 -> 분기 (bit2 안 켜짐)
329       ori  x10, x10, 4
330  L3:  bne  x2, x3, L4       # 5 != 5 거짓 -> 진행 (bit3 켜짐)
331       ori  x10, x10, 8
332  L4:  sw   x10, 0(x1)       # x10 = 2 | 8 = 0x0a
333       .word 0               # 0x00040038
334       """,
335       stdout=line(0x00040038, 0),
336       expect_words={0: 0x0000000A})
337
338  case("14_blt_bge_signed",
339       "blt/bge 는 부호 있는 비교. 0xffffffff(-1) < 1 이므로 blt 분기. INT_MIN/INT_MAX, 같은 값. "
340       "(부호 없는 비교로 잘못 구현하면 결과가 0x99 가 된다.)",
341       """
342       lui  x1, 0x10000
343       addi x2, x0, -1       # 0xffffffff = -1
344       addi x3, x0, 1        # 1
345       lui  x4, 0x80000      # 0x80000000 = INT_MIN
346       addi x5, x4, -1       # 0x7fffffff = INT_MAX
347       addi x10, x0, 0
348       blt  x2, x3, A        # -1 < 1 참 -> 분기
349       ori  x10, x10, 1
350  A:   bge  x2, x3, B        # -1 >= 1 거짓 -> 진행: bit1
351       ori  x10, x10, 2
352  B:   blt  x3, x2, C        # 1 < -1 거짓 -> 진행: bit2
353       ori  x10, x10, 4
354  C:   bge  x3, x2, D        # 1 >= -1 참 -> 분기
355       ori  x10, x10, 8
356  D:   blt  x4, x5, E        # INT_MIN < INT_MAX 참 -> 분기
357       ori  x10, x10, 16
358  E:   bge  x4, x5, F        # INT_MIN >= INT_MAX 거짓 -> 진행: bit5
359       ori  x10, x10, 32
360  F:   bge  x3, x3, G        # 1 >= 1 참(같음) -> 분기
361       ori  x10, x10, 64
362  G:   blt  x3, x3, H        # 1 < 1 거짓 -> 진행: bit7
363       ori  x10, x10, 128
364  H:   sw   x10, 0(x1)       # 2 + 4 + 32 + 128 = 166 = 0xa6
365       .word 0               # 0x0004005c
366       """,
367       stdout=line(0x0004005C, 0),
368       expect_words={0: 0x000000A6})
369
370  # 14: B형 즉시값의 모든 비트 위치 확인 (+2048: imm[11]=bit7, +2032: imm[10:4], -4096: imm[12]만)
371  _far = ["lui  x1, 0x10000",
372          "addi x10, x0, 0",
373          "beq  x0, x0, 16          # 0x00040008 -> 0x00040018 (start)",
374          "ori  x10, x10, 4         # 0x0004000c: -4096 분기의 도착점",
375          "sw   x10, 0(x1)          # 0x00040010",
376          ".word 0                  # 0x00040014: 종료 지점",
377          "beq  x0, x0, 2048        # 0x00040018 -> 0x00040818 (imm[11] 이 bit 7 로 감)"]
378  _far += ["ori  x10, x10, 1        # 건너뛰어야 하는 영역"] * 511       # 0x0004001c ~ 0x00040814
```

```
_far += ["ori  x10, x10, 2         # 0x00040818",
         "beq  x0, x0, 2032        # 0x0004081c -> 0x0004100c"]
_far += ["ori  x10, x10, 1         # 건너뛰어야 하는 영역"] * 507       # 0x00040820 ~ 0x00041008
_far += ["beq  x0, x0, -4096       # 0x0004100c -> 0x0004000c (imm[12] 만 1)",
         ".word 0                  # 0x00041010: text 마지막 워드 (도달하지 않음)"]
case("15_branch_far_offsets",
     "B형 즉시값 비트 배치 확인: +2048(imm[11] 이 명령어 bit7), +2032, -4096(최소값). "
     "건너뛴 영역이 하나라도 실행되면 bit0 이 켜진다. 기대 x10 = 2 | 4 = 6.",
     "\n".join(_far),
     stdout=line(0x00040014, 0),
     expect_words={0: 0x00000006})

case("16_branch_over_invalid",
     "invalid 판정은 적재 시점이 아니라 fetch 시점에 한다. 분기로 건너뛴 invalid 워드는 종료를 일으키지 않는다.",
     """
     lui  x1, 0x10000
     addi x2, x0, 1
     beq  x0, x0, skip     # 항상 분기 -> 0x00040014
     .word 0x00000000      # fetch 되지 않음
     .word 0xffffffff      # fetch 되지 않음
skip: sw  x2, 0(x1)        # [0] = 1
     jal  x0, 0            # 0x00040018: 0x0000006f (jal 은 미지원 -> 종료, 실행되면 무한 루프)
     .word 0               # text 마지막 워드
     """,
     stdout=line(0x00040018, 0x0000006F),
     expect_words={0: 0x00000001})

case("17_loop_sum",
     "뒤로 가는 분기(음수 오프셋 -16)로 반복. data 의 1~10 합 = 55 (0x37). 실행 명령어 수 3+5*10+2 = 55.",
     """
     lui  x1, 0x10000      # 포인터
     addi x2, x0, 10       # 반복 횟수
     addi x3, x0, 0        # 합
loop: lw  x4, 0(x1)        # 0x0004000c
     add  x3, x3, x4
     addi x1, x1, 4
     addi x2, x2, -1
     bne  x2, x0, loop     # 0x0004001c -> 0x0004000c (오프셋 -16)
     lui  x5, 0x10000
     sw   x3, 64(x5)       # [0x10000040] = 55
     .word 0               # 0x00040028
     """,
     data=words(*range(1, 11)),
     stdout=line(0x00040028, 0),
     expect_words={64: 55})

case("18_fibonacci",
     "피보나치 F(0)~F(19) 를 데이터 영역에 저장. 레지스터 복사(addi rd, rs, 0)와 반복.",
     """
     lui  x1, 0x10000
     addi x2, x0, 0        # a = F(0)
     addi x3, x0, 1        # b = F(1)
     addi x4, x0, 20       # 20 개
loop: sw  x2, 0(x1)
     add  x5, x2, x3       # a + b
     addi x2, x3, 0        # a = b
     addi x3, x5, 0        # b = a + b
     addi x1, x1, 4
     addi x4, x4, -1
     bne  x4, x0, loop
     .word 0               # 0x0004002c
     """,
     stdout=line(0x0004002C, 0),
     expect_words={4 * i: v for i, v in enumerate(
         [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597,
          2584, 4181])})

case("19_multiply_signed",
     "반복 덧셈 곱셈 (-3) * 7 = -21 = 0xffffffeb. blt x0, x3 (0 < count, 부호 비교) 루프.",
     """
     lui  x1, 0x10000
     lw   x2, 0(x1)        # -3 = 0xfffffffd
     lw   x3, 4(x1)        # 7
     addi x4, x0, 0
loop: add x4, x4, x2       # 0x00040010
     addi x3, x3, -1
```

```
     blt  x0, x3, loop     # 0 < x3 이면 반복
     sw   x4, 8(x1)        # -21 = 0xffffffeb
     .word 0               # 0x00040020
     """,
     data=words(0xFFFFFFFD, 7),
     stdout=line(0x00040020, 0),
     expect_words={8: 0xFFFFFFEB})

case("20_bubble_sort",
     "부호 있는 정수 8개 버블 정렬(bge 부호 비교, lw/sw, 이중 루프). "
     "[5,-3,12,0,-7,8,1,-1] -> [-7,-3,-1,0,1,5,8,12].",
     """
     lui  x1, 0x10000      # 배열 시작
     addi x2, x0, 7        # 바깥 루프: 남은 비교 횟수 i = n-1
outer: addi x3, x1, 0      # p = &a[0]
     addi x4, x2, 0        # j = i
inner: lw  x5, 0(x3)       # a[k]
     lw   x6, 4(x3)        # a[k+1]
     bge  x6, x5, noswap   # a[k+1] >= a[k] 이면 교환 안 함 (부호 비교)
     sw   x6, 0(x3)
     sw   x5, 4(x3)
noswap: addi x3, x3, 4
     addi x4, x4, -1
     bne  x4, x0, inner
     addi x2, x2, -1
     bne  x2, x0, outer
     .word 0               # 0x00040038
     """,
     data=words(5, -3, 12, 0, -7, 8, 1, -1),
     stdout=line(0x00040038, 0),
     expect_words={4 * i: v & 0xFFFFFFFF for i, v in enumerate([-7, -3, -1, 0, 1, 5, 8, 12])})

_regs = ["lui  x31, 0x10000       # 기준 주소 레지스터 x31 (rs1 = 31)"]
_regs += ["addi x%d, x0, %d" % (i, i) for i in range(1, 31)]          # rd = 1 ~ 30
_regs += ["sw   x%d, %d(x31)" % (i, 4 * i) for i in range(1, 31)]      # rs2 = 1 ~ 30
_regs += ["sw   x31, 124(x31)       # [124] = 0x10000000 (rs2 = 31)",
          ".word 0                  # 0x000400f8"]
case("21_all_registers",
     "rd/rs1/rs2 필드 5비트 해독: x1~x31 을 모두 쓰고 저장. 기대 [4*i] = i, [124] = 0x10000000.",
     "\n".join(_regs),
     stdout=line(0x000400F8, 0),
     expect_words=dict([(4 * i, i) for i in range(1, 31)] + [(124, 0x10000000)]))

# ---------------------------------------------------------------------------
# 5. invalid instruction 판별: opcode, funct3, funct7 까지 확인
#    공통 구조: 유효 명령어 실행 -> 0x00040014 의 invalid 에서 종료 -> 뒤의 sw 는 실행되지 않음
# ---------------------------------------------------------------------------
_INVALID_TEMPLATE = """
     lui  x1, 0x10000
     addi x2, x0, 3
     addi x3, x0, 4
     add  x4, x2, x3       # 7
     sw   x4, 0(x1)        # [0] = 7 (0xaaaaaaaa 를 덮어씀)
     {invalid}             # 0x00040014: invalid -> 실행하지 않고 종료
     sw   x2, 4(x1)        # 실행되면 안 됨 ([4] 는 0xcafebabe 그대로)
     .word 0
"""

# (이름, 어셈블리, 손으로 계산한 명령어 값, 목적)
_INVALID_CASES = [
    ("22_invalid_mul", "mul x5, x2, x3", 0x023102B3,
     "add 와 opcode·funct3 가 같고 funct7 만 0000001 (RV32M mul). funct7 검사 필수."),
    ("23_invalid_sra", "sra x5, x2, x3", 0x40315283,
     "funct7=0100000(sub 와 같음) + funct3=101. funct7 만 보고 sub 로 판단하면 안 됨."),
    ("24_invalid_and_funct7", ".word 0x403172b3", 0x403172B3,
     "and(funct3=111) 에 funct7=0100000 을 붙인 값. and/or 도 funct7=0 확인 필요."),
    ("25_invalid_xor", "xor x5, x2, x3", 0x003142B3,
     "R형 opcode 이지만 funct3=100 (지원하지 않는 xor)."),
    ("26_invalid_slli", "slli x5, x2, 3", 0x00311293,
     "I형 연산 opcode(0010011)이지만 funct3=001 (slli)."),
    ("27_invalid_lb", "lb x5, 0(x1)", 0x00008283,
     "load opcode 이지만 funct3=000 (lb). lw 는 funct3=010 만."),
    ("28_invalid_sh", "sh x2, 4(x1)", 0x00209223,
     "store opcode 이지만 funct3=001 (sh). sw 는 funct3=010 만."),
    ("29_invalid_bltu", "bltu x2, x3, 8", 0x00316463,
     "branch opcode 이지만 funct3=110 (bltu, 부호 없는 비교). blt 로 처리하면 안 됨."),
```

```
    ("30_invalid_jal", "jal x0, 0", 0x0000006F,
     "지원하지 않는 opcode 1101111 (jal). 실행되면 무한 루프."),
    ("31_invalid_ecall", "ecall", 0x00000073,
     "시스템 호출은 지원 제외 (opcode 1110011)."),
    ("32_invalid_all_ones", ".word 0xffffffff", 0xFFFFFFFF,
     "opcode 1111111 은 어떤 지원 명령어도 아님."),
]
for _name, _asm, _word, _purpose in _INVALID_CASES:
    case(_name, _purpose, _INVALID_TEMPLATE.format(invalid=_asm),
         data=words(0xAAAAAAAA, 0xCAFEBABE),
         stdout=line(0x00040014, _word),
         expect_words={0: 7})

# ---------------------------------------------------------------------------
# 6. 실행 한도 (정상 명령어 최대 1,000,000 개)
# ---------------------------------------------------------------------------
case("33_limit_exact",
     "정확히 1,000,000 개 실행 후 invalid 를 fetch -> 정상 종료. "
     "실행 수 = 2 + 2 * 499999 = 1,000,000 (x1 = 0x7a11f = 499999).",
     """
     lui  x1, 0x7a         # 0x0007a000 = 499712
     addi x1, x1, 0x11f    # + 287 = 499999
loop: addi x1, x1, -1      # 0x00040008
     bne  x1, x0, loop     # 0x0004000c
     .word 0               # 0x00040010: 1,000,001 번째 fetch -> invalid -> 정상 종료
     """,
     stdout=line(0x00040010, 0))

case("34_limit_plus_one",
     "필요한 실행 수가 정확히 1,000,001 개(= 3 + 2 * 499999). 1,000,000 번째로 addi 를 실행한 뒤 "
     "다음 fetch 는 지원 명령어 bne -> 실행하지 않고 비정상 종료(종료 코드 2). "
     "한도 비교를 > 로 잘못 쓰면 bne 까지 실행되어 정상 종료가 되어 버린다.",
     """
     lui  x1, 0x7a
     addi x1, x1, 0x11f    # 499999
     nop                   # 실행 수 +1
loop: addi x1, x1, -1      # 1,000,000 번째 실행 = 마지막 반복의 addi
     bne  x1, x0, loop     # 1,000,001 번째 -> 실행하지 않고 비정상 종료
     .word 0
     """,
     exit_code=2)

case("35_limit_exceeded",
     "x1 = 500000 이면 필요한 실행 수가 1,000,002 개. 1,000,000 개 실행 뒤 다음 fetch(addi)가 "
     "지원 명령어이므로 실행하지 않고 비정상 종료(종료 코드 2, 표준 출력 없음).",
     """
     lui  x1, 0x7a         # 0x0007a000
     addi x1, x1, 0x120    # + 288 = 0x7a120 = 500000
loop: addi x1, x1, -1
     bne  x1, x0, loop
     .word 0
     """,
     exit_code=2)

case("36_infinite_loop",
     "자기 자신으로 분기하는 무한 루프 -> 실행 한도 초과로 비정상 종료(종료 코드 2).",
     """
loop: beq x0, x0, loop     # 오프셋 0
     .word 0
     """,
     exit_code=2)

# ---------------------------------------------------------------------------
# 7. 명세 범위 밖 입력에 대한 방어 동작 (채점 대상 아님, 우리 팀의 설계 결정)
# ---------------------------------------------------------------------------
case("37_err_lw_out_of_range",
     "데이터 영역 밖(0x10000400) lw -> 표준 오류 메시지, 종료 코드 3, 표준 출력 없음.",
     """
     lui  x1, 0x10000
     lw   x2, 1024(x1)     # 0x10000400: 범위 밖
     .word 0
     """,
     exit_code=3)

case("38_err_sw_misaligned",
     "4바이트 정렬되지 않은 sw(0x10000002) -> 종료 코드 3.",
```

```
    """
    lui  x1, 0x10000
    sw   x0, 2(x1)
    .word 0
    """,
    exit_code=3)

case("39_err_pc_out_of_text",
    "적재된 text 밖으로 분기(0x00040010) -> fetch 불가, 종료 코드 3.",
    """
    beq  x0, x0, 16       # 0x00040000 -> 0x00040010 (text 는 8 바이트뿐)
    .word 0
    """,
    exit_code=3)

case("40_err_sw_to_text",
    "text segment 쓰기 불가: sw 주소가 0x00040000 -> 종료 코드 3.",
    """
    lui  x1, 0x40         # 0x00040000
    sw   x0, 0(x1)
    .word 0
    """,
    exit_code=3)


def expected_data_out(c):
    """data.bin(1,024 바이트까지 0 으로 채움)에 기대 변경을 적용한 data.out 기대값."""
    image = bytearray(DATA_SIZE)
    image[0:min(len(c["data"]), DATA_SIZE)] = c["data"][0:DATA_SIZE]
    for offset, value in c["expect_words"].items():
        image[offset:offset + 4] = struct.pack("<I", value)
    for offset, hexstr in c["expect_bytes"].items():
        raw = bytes.fromhex(hexstr)
        image[offset:offset + len(raw)] = raw
    return bytes(image)
```

tests/gen_tests.py (114줄)

```
#!/usr/bin/env python3
"""tests/cases.py 의 정의로 tests/cases/<이름>/ 테스트 입출력 파일을 생성한다.

생성 파일 (케이스마다)
  program.s            주소 · 기계어 · 어셈블리 목록과 테스트 목적
  text.bin, data.bin   시뮬레이터 입력 (헤더 없는 바이너리)
  expected_stdout.txt  기대 표준 출력 (비정상 종료 케이스는 빈 파일)
  expected_data.out    기대 data.out 1,024 바이트 (정상 종료 케이스만)
  expected_exit_code.txt 기대 종료 코드
그리고 tests/cases/INDEX.md (전체 목록)

사용법:  python tests/gen_tests.py
"""

import os
import shutil
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from asm import TEXT_BASE, assemble, listing, words_to_bytes  # noqa: E402
from cases import CASES, expected_data_out  # noqa: E402

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(HERE, "cases")


def write_bytes(path, data):
    with open(path, "wb") as f:
        f.write(data)


def write_text_lf(path, text):
    """줄바꿈 변환 없이 LF 그대로 저장 (Windows 에서도 WWrWWn 이 되지 않도록 바이너리로 기록)."""
    write_bytes(path, text.encode("utf-8"))


def check_case(c, words):
    """생성 전 자기 점검: 손으로 계산한 값과 어셈블러 결과를 교차 확인."""
    text = words_to_bytes(words)
    if c["check_text"] is not None and text != c["check_text"]:
        raise SystemExit("%s: text.bin differs from spec bytesWn  got  %sWn  want %s"
                         % (c["name"], text.hex(), c["check_text"].hex()))
    if words[-1] != 0:
        raise SystemExit("%s: text.bin must end with 0x00000000 (spec 6)" % c["name"])
    if len(c["data"]) > 1024:
        raise SystemExit("%s: data.bin larger than 1024 bytes" % c["name"])
    if c["exit_code"] == 0:
        # 기대 출력의 명령어 값 == 그 주소에 어셈블된 값 (손 계산 vs 인코더)
        parts = c["stdout"].split()
        pc, word = int(parts[1], 16), int(parts[4], 16)
        index = (pc - TEXT_BASE) // 4
        if words[index] != word:
            raise SystemExit("%s: expected word 0x%08x but assembled 0x%08x at 0x%08x"
                             % (c["name"], word, words[index], pc))
        if not c["stdout"].endswith("Wn") or "Wr" in c["stdout"]:
            raise SystemExit("%s: stdout must end with a single LF" % c["name"])


def program_listing(c):
    lines = ["# %s" % c["name"], "# 목적: %s" % c["purpose"], "#",
             "# 주소        기계어      어셈블리"]
    for pc, word, raw in listing(c["source"]):
        lines.append("0x%08x  0x%08x  %s" % (pc, word, raw))
    return "Wn".join(lines) + "Wn"


def main():
    # cases.py 에 정의된 케이스 디렉터리만 다시 만든다.
    # 팀원이 손으로 추가한 tests/cases/<다른 이름>/ 은 지우지 않는다 (run_tests.py 는 그것도 실행한다).
    os.makedirs(OUT_DIR, exist_ok=True)
    defined = {c["name"] for c in CASES}
    for extra in sorted(os.listdir(OUT_DIR)):
        if os.path.isdir(os.path.join(OUT_DIR, extra)) and extra not in defined:
```

```
            print("keeping manually added case (not in cases.py): " + extra)

    index = ["# 자체 테스트 목록", "",
             "`python tests/gen_tests.py` 가 `tests/cases.py` 로부터 생성한 파일이다. "
             "기대 결과는 `tests/cases.py` 의 주석에 도출 근거가 있다.", "",
             "| 케이스 | 목적 | 기대 표준 출력 / 종료 코드 |",
             "|---|---|---|"]
    names = set()
    for c in CASES:
        if c["name"] in names:
            raise SystemExit("duplicate case name: " + c["name"])
        names.add(c["name"])

        words = assemble(c["source"])
        check_case(c, words)

        d = os.path.join(OUT_DIR, c["name"])
        if os.path.isdir(d):
            shutil.rmtree(d)        # 이전 생성물(예: 정상 -> 비정상으로 바뀐 케이스의 expected_data.out) 제거
        os.makedirs(d)
        write_bytes(os.path.join(d, "text.bin"), words_to_bytes(words))
        write_bytes(os.path.join(d, "data.bin"), c["data"])
        write_text_lf(os.path.join(d, "expected_stdout.txt"), c["stdout"])
        write_text_lf(os.path.join(d, "expected_exit_code.txt"), "%dWn" % c["exit_code"])
        write_text_lf(os.path.join(d, "program.s"), program_listing(c))
        if c["exit_code"] == 0:
            image = expected_data_out(c)
            assert len(image) == 1024
            write_bytes(os.path.join(d, "expected_data.out"), image)

        expect = ("`%s`" % c["stdout"].strip()) if c["exit_code"] == 0 else W
            "표준 출력 없음, 종료 코드 %d" % c["exit_code"]
        index.append("| %s | %s | %s |" % (c["name"], c["purpose"].replace("|", "WW|"), expect))

    write_text_lf(os.path.join(OUT_DIR, "INDEX.md"), "Wn".join(index) + "Wn")
    print("generated %d test cases in %s" % (len(CASES), OUT_DIR))


if __name__ == "__main__":
    main()
```

tests/run_tests.py (151줄)

```
#!/usr/bin/env python3
"""tests/cases/*/ 의 모든 테스트를 실제 실행 방식 그대로(별도 프로세스) 돌려 바이트 단위로 비교한다.

각 테스트마다
  1) 새 임시 디렉터리를 현재 작업 디렉터리로 하여 시뮬레이터 실행
     (정상 종료 케이스는 쓰레기 값 data.out 2,048 바이트를 미리 만들어 덮어쓰기도 확인)
  2) 종료 코드 비교
  3) 표준 출력 바이트 비교 (줄바꿈 WWn 하나까지 정확히)
  4) 정상 종료면 data.out 크기 1,024 와 내용 바이트 비교,
     비정상 종료면 data.out 이 만들어지지 않았는지 확인

사용법
  python tests/run_tests.py                    # 기본: 현재 파이썬으로 sim.py 실행
  python tests/run_tests.py --sim "python3 /path/sim.py"   # 다른 실행 명령 지정
  python tests/run_tests.py 01_spec_example 13_blt_bge_signed  # 일부만
"""

import argparse
import os
import shlex
import subprocess
import sys
import tempfile
import time

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
CASES_DIR = os.path.join(HERE, "cases")


def read_bytes(path):
    with open(path, "rb") as f:
        return f.read()


def first_diff(a, b):
    for i in range(min(len(a), len(b))):
        if a[i] != b[i]:
            return i
    return min(len(a), len(b))


def run_case(sim_cmd, case_dir):
    """실패 사유 목록을 반환한다 (빈 목록 = 통과)."""
    text_bin = os.path.join(case_dir, "text.bin")
    data_bin = os.path.join(case_dir, "data.bin")
    want_stdout = read_bytes(os.path.join(case_dir, "expected_stdout.txt"))
    want_exit = int(read_bytes(os.path.join(case_dir, "expected_exit_code.txt")))
    want_data_path = os.path.join(case_dir, "expected_data.out")

    problems = []
    with tempfile.TemporaryDirectory() as work:
        out_path = os.path.join(work, "data.out")
        if want_exit == 0:
            with open(out_path, "wb") as f:      # 기존 data.out 덮어쓰기 확인용
                f.write(b"Wxee" * 2048)
        proc = subprocess.run(sim_cmd + [text_bin, data_bin], cwd=work,
                              stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=120)

        if proc.returncode != want_exit:
            problems.append("exit code %d, expected %d (stderr: %s)"
                            % (proc.returncode, want_exit,
                               proc.stderr.decode(errors="replace").strip()))
        if proc.stdout != want_stdout:
            problems.append("stdout %r, expected %r" % (proc.stdout, want_stdout))

        if want_exit == 0:
            if not os.path.exists(out_path):
                problems.append("data.out not created")
            else:
                got = read_bytes(out_path)
                want = read_bytes(want_data_path)
                if len(got) != 1024:
                    problems.append("data.out size %d, expected 1024" % len(got))
```

```
                if got != want:
                    i = first_diff(got, want)
                    problems.append("data.out differs at offset 0x%03x (addr 0x%08x): "
                                    "got %s expected %s"
                                    % (i, 0x10000000 + i, got[i:i + 4].hex(" "),
                                       want[i:i + 4].hex(" ")))
        elif os.path.exists(out_path):
            problems.append("data.out must not be created on abnormal termination")
    return problems


def run_cli_checks(sim_cmd):
    """인수 개수·파일 없음 오류 처리 확인 (종료 코드 1, 표준 출력 없음)."""
    results = []
    with tempfile.TemporaryDirectory() as work:
        for label, args in [("no arguments", []),
                            ("one argument", ["text.bin"]),
                            ("three arguments", ["a", "b", "c"]),
                            ("missing input file", ["no_such_text.bin", "no_such_data.bin"])]:
            proc = subprocess.run(sim_cmd + args, cwd=work, stdout=subprocess.PIPE,
                                  stderr=subprocess.PIPE, timeout=60)
            problems = []
            if proc.returncode != 1:
                problems.append("exit code %d, expected 1" % proc.returncode)
            if proc.stdout:
                problems.append("stdout must be empty, got %r" % proc.stdout)
            if not proc.stderr:
                problems.append("stderr should explain the error")
            results.append(("cli: " + label, problems))
    return results


def main():
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("--sim", help="simulator command (default: current python + sim.py)")
    parser.add_argument("cases", nargs="*", help="case names to run (default: all)")
    args = parser.parse_args()

    if args.sim:
        # Windows 경로의 백슬래시가 사라지지 않도록 Windows 에서는 non-POSIX 규칙으로 나누고 따옴표만 벗긴다.
        sim_cmd = [t[1:-1] if len(t) > 1 and t[0] == t[-1] == '"' else t
                   for t in shlex.split(args.sim, posix=(os.name != "nt"))]
    else:
        sim_cmd = [sys.executable, os.path.join(ROOT, "sim.py")]

    if not os.path.isdir(CASES_DIR):
        sys.exit("tests/cases not found. run: python tests/gen_tests.py")
    names = args.cases or sorted(n for n in os.listdir(CASES_DIR)
                                 if os.path.isdir(os.path.join(CASES_DIR, n)))

    failed = 0
    total = 0
    for name in names:
        start = time.time()
        problems = run_case(sim_cmd, os.path.join(CASES_DIR, name))
        total += 1
        status = "PASS" if not problems else "FAIL"
        print("%s  %-28s %6.2fs" % (status, name, time.time() - start))
        for p in problems:
            print("      - " + p)
        failed += bool(problems)

    if not args.cases:
        for label, problems in run_cli_checks(sim_cmd):
            total += 1
            print("%s  %s" % ("PASS" if not problems else "FAIL", label))
            for p in problems:
                print("      - " + p)
            failed += bool(problems)

    print("Wn%d/%d passed" % (total - failed, total))
    sys.exit(1 if failed else 0)


```

```
150 if __name__ == "__main__":
151     main()
```

tests/test_units.py (280줄)

```python
#!/usr/bin/env python3
"""단위 테스트: 비트 처리, 즉시값, 해독/invalid 판별, 메모리, CPU 실행 규칙.

사용법:  python tests/test_units.py      (표준 라이브러리 unittest 만 사용)
"""

import os
import random
import sys
import unittest

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(HERE))   # rvsim 패키지
sys.path.insert(0, HERE)                    # asm.py

from rvsim.bits import get_bits, sign_extend, to_signed32, to_u32  # noqa: E402
from rvsim.cpu import CPU, ExecutionLimitExceeded  # noqa: E402
from rvsim.decoder import decode, imm_b, imm_i, imm_s, imm_u  # noqa: E402
from rvsim.memory import (DATA_BASE, TEXT_BASE, Memory, SimError,  # noqa: E402
                          read_word_le, write_word_le)
import asm  # noqa: E402
from sim import format_termination_line  # noqa: E402


def make_cpu(source, data=b""):
    return CPU(Memory(asm.words_to_bytes(asm.assemble(source)), data))


class BitsTest(unittest.TestCase):
    def test_to_u32(self):
        self.assertEqual(to_u32(0xFFFFFFFF + 1), 0)
        self.assertEqual(to_u32(-1), 0xFFFFFFFF)
        self.assertEqual(to_u32(-2048), 0xFFFFF800)
        self.assertEqual(to_u32(0x1_2345_6789), 0x23456789)

    def test_to_signed32(self):
        self.assertEqual(to_signed32(0xFFFFFFFF), -1)
        self.assertEqual(to_signed32(0x80000000), -2147483648)
        self.assertEqual(to_signed32(0x7FFFFFFF), 2147483647)
        self.assertEqual(to_signed32(1), 1)
        self.assertEqual(to_signed32(0), 0)

    def test_sign_extend(self):
        self.assertEqual(sign_extend(0xFFF, 12), -1)
        self.assertEqual(sign_extend(0x800, 12), -2048)
        self.assertEqual(sign_extend(0x7FF, 12), 2047)
        self.assertEqual(sign_extend(0x000, 12), 0)
        self.assertEqual(sign_extend(0x1000, 13), -4096)
        self.assertEqual(sign_extend(0x0FFE, 13), 4094)

    def test_get_bits(self):
        self.assertEqual(get_bits(0x00700113, 6, 0), 0x13)
        self.assertEqual(get_bits(0x00700113, 31, 20), 7)
        self.assertEqual(get_bits(0x80000000, 31, 31), 1)


class ImmediateTest(unittest.TestCase):
    def test_i_boundaries(self):
        for imm in (-2048, -1, 0, 1, 2047):
            self.assertEqual(imm_i(asm.enc_i(imm, 1, 0, 2, 0x13)), imm)

    def test_s_boundaries(self):
        for imm in (-2048, -516, -1, 0, 1, 31, 32, 2044, 2047):
            self.assertEqual(imm_s(asm.enc_s(imm, 3, 1, 0b010, 0x23)), imm)

    def test_b_boundaries(self):
        for off in (-4096, -2048, -16, -4, -2, 0, 2, 4, 2032, 2048, 4094):
            self.assertEqual(imm_b(asm.enc_b(off, 2, 1, 0, 0x63)), off)

    def test_u(self):
        self.assertEqual(imm_u(asm.enc_u(0xFFFFF, 1, 0x37)), 0xFFFFF000)
        self.assertEqual(imm_u(0x100000B7), 0x10000000)

    def test_random_roundtrip(self):
```

```
        rng = random.Random(1234)
        for _ in range(2000):
            i = rng.randint(-2048, 2047)
            b = rng.randrange(-4096, 4095, 2)
            self.assertEqual(imm_i(asm.enc_i(i, rng.randrange(32), 0, rng.randrange(32), 0x13)), i)
            self.assertEqual(imm_s(asm.enc_s(i, rng.randrange(32), rng.randrange(32), 2, 0x23)), i)
            self.assertEqual(imm_b(asm.enc_b(b, rng.randrange(32), rng.randrange(32), 0, 0x63)), b)


class DecodeTest(unittest.TestCase):
    def check(self, word, name, rd=0, rs1=0, rs2=0, imm=0):
        inst = decode(word)
        self.assertIsNotNone(inst, "0x%08x should be valid" % word)
        self.assertEqual((inst.name, inst.rd, inst.rs1, inst.rs2, inst.imm),
                         (name, rd, rs1, rs2, imm), "0x%08x" % word)

    def test_spec_example_words(self):
        self.check(0x100000B7, "lui", rd=1, imm=0x10000000)
        self.check(0x00700113, "addi", rd=2, rs1=0, imm=7)
        self.check(0x00500193, "addi", rd=3, rs1=0, imm=5)
        self.check(0x00310233, "add", rd=4, rs1=2, rs2=3)
        self.check(0x0040A023, "sw", rs1=1, rs2=4, imm=0)
        self.assertIsNone(decode(0x00000000))

    def test_all_supported_mnemonics(self):
        cases = [
            ("add x5, x6, x7", ("add", 5, 6, 7, 0)),
            ("sub x31, x30, x29", ("sub", 31, 30, 29, 0)),
            ("and x1, x2, x3", ("and", 1, 2, 3, 0)),
            ("or x4, x5, x6", ("or", 4, 5, 6, 0)),
            ("addi x1, x2, -5", ("addi", 1, 2, 0, -5)),
            ("andi x3, x4, -1", ("andi", 3, 4, 0, -1)),
            ("ori x5, x6, 2047", ("ori", 5, 6, 0, 2047)),
            ("lui x7, 0xfffff", ("lui", 7, 0, 0, 0xFFFFF000)),
            ("lw x8, -4(x9)", ("lw", 8, 9, 0, -4)),
            ("sw x10, 2044(x11)", ("sw", 0, 11, 10, 2044)),
            ("beq x1, x2, -8", ("beq", 0, 1, 2, -8)),
            ("bne x3, x4, 12", ("bne", 0, 3, 4, 12)),
            ("blt x5, x6, -4096", ("blt", 0, 5, 6, -4096)),
            ("bge x7, x8, 4094", ("bge", 0, 7, 8, 4094)),
        ]
        for src, want in cases:
            word = asm.assemble(src)[0]
            inst = decode(word)
            self.assertIsNotNone(inst, src)
            self.assertEqual((inst.name, inst.rd, inst.rs1, inst.rs2, inst.imm), want, src)

    def test_unsupported_instructions_are_invalid(self):
        unsupported = [
            "xor x1, x2, x3", "sll x1, x2, x3", "srl x1, x2, x3", "sra x1, x2, x3",
            "slt x1, x2, x3", "sltu x1, x2, x3", "mul x1, x2, x3",
            "slti x1, x2, 5", "sltiu x1, x2, 5", "xori x1, x2, 5",
            "slli x1, x2, 3", "srli x1, x2, 3", "srai x1, x2, 3",
            "lb x1, 0(x2)", "lh x1, 0(x2)", "lbu x1, 0(x2)", "lhu x1, 0(x2)",
            "sb x1, 0(x2)", "sh x1, 0(x2)", "bltu x1, x2, 8", "bgeu x1, x2, 8",
            "jal x1, 8", "jalr x1, 0(x2)", "auipc x1, 0x10", "ecall", "ebreak",
        ]
        for src in unsupported:
            word = asm.assemble(src)[0]
            self.assertIsNone(decode(word), "%s (0x%08x) must be invalid" % (src, word))

    def test_wrong_funct7_is_invalid(self):
        add = asm.assemble("add x1, x2, x3")[0]
        for funct7 in range(128):
            word = (add & 0x01FFFFFF) | (funct7 << 25)
            expect_valid = funct7 in (0b0000000, 0b0100000)   # add / sub
            self.assertEqual(decode(word) is not None, expect_valid, "funct7=%d" % funct7)
        for src in ("and x1, x2, x3", "or x1, x2, x3"):
            base = asm.assemble(src)[0]
            for funct7 in range(1, 128):
                self.assertIsNone(decode((base & 0x01FFFFFF) | (funct7 << 25)))

    def test_every_funct3_per_opcode(self):
        valid = {0x13: {0, 6, 7}, 0x03: {2}, 0x23: {2}, 0x63: {0, 1, 4, 5}}
        for opcode, ok in valid.items():
            for funct3 in range(8):
```

```
                word = (funct3 << 12) | opcode
                self.assertEqual(decode(word) is not None, funct3 in ok,
                                 "opcode=0x%02x funct3=%d" % (opcode, funct3))

    def test_only_six_opcodes_are_valid(self):
        valid_opcodes = {0x33, 0x13, 0x03, 0x23, 0x63, 0x37}
        for opcode in range(128):
            # funct3/funct7 = 0 이면 add/addi/beq/lui 는 유효, lw/sw 는 funct3=2 필요
            word = opcode | (2 << 12 if opcode in (0x03, 0x23) else 0)
            self.assertEqual(decode(word) is not None, opcode in valid_opcodes,
                             "opcode=0x%02x" % opcode)

    def test_random_words_match_reference_rule(self):
        """무작위 32비트 값에 대해 decode 의 유효 판정이 명세 표와 일치하는지 (독립 규칙으로 검사)."""
        rng = random.Random(42)
        for _ in range(200000):
            w = rng.getrandbits(32)
            op, f3, f7 = w & 0x7F, (w >> 12) & 7, w >> 25
            ref = ((op == 0x33 and (f3, f7) in ((0, 0), (0, 0x20), (7, 0), (6, 0)))
                   or (op == 0x13 and f3 in (0, 6, 7))
                   or (op == 0x03 and f3 == 2)
                   or (op == 0x23 and f3 == 2)
                   or (op == 0x63 and f3 in (0, 1, 4, 5))
                   or op == 0x37)
            self.assertEqual(decode(w) is not None, ref, "0x%08x" % w)


class MemoryTest(unittest.TestCase):
    def test_little_endian_read(self):
        self.assertEqual(read_word_le(bytes([0x78, 0x56, 0x34, 0x12]), 0), 0x12345678)
        self.assertEqual(read_word_le(bytes.fromhex("b7000010"), 0), 0x100000B7)

    def test_little_endian_write(self):
        buf = bytearray(4)
        write_word_le(buf, 0, 0x12345678)
        self.assertEqual(bytes(buf), bytes([0x78, 0x56, 0x34, 0x12]))

    def test_data_zero_init_and_load(self):
        m = Memory(b"Wx00" * 4, b"Wx01Wx02Wx03")
        self.assertEqual(len(m.data), 1024)
        self.assertEqual(m.load_word(DATA_BASE), 0x00030201)
        self.assertEqual(m.load_word(DATA_BASE + 1020), 0)

    def test_data_bounds_and_alignment(self):
        m = Memory(b"Wx00" * 4, b"")
        m.store_word(DATA_BASE + 1020, 0xA1B2C3D4)              # 마지막 워드 OK
        self.assertEqual(m.data[1020:1024], bytes.fromhex("d4c3b2a1"))
        for bad in (DATA_BASE - 4, DATA_BASE + 1024, DATA_BASE + 2, DATA_BASE + 1021,
                    TEXT_BASE, 0, 0xFFFFFFFC):
            with self.assertRaises(SimError):
                m.load_word(bad)
            with self.assertRaises(SimError):
                m.store_word(bad, 0)

    def test_fetch_bounds(self):
        m = Memory(bytes.fromhex("13000000 00000000"), b"")
        self.assertEqual(m.fetch(TEXT_BASE), 0x00000013)
        self.assertEqual(m.fetch(TEXT_BASE + 4), 0)
        for bad in (TEXT_BASE + 8, TEXT_BASE - 4, TEXT_BASE + 2, DATA_BASE):
            with self.assertRaises(SimError):
                m.fetch(bad)


class CpuTest(unittest.TestCase):
    def test_x0_is_always_zero(self):
        cpu = make_cpu("addi x0, x0, 1Wnlui x0, 0xfffffWnadd x0, x0, x0Wn.word 0")
        cpu.run()
        self.assertEqual(cpu.regs[0], 0)

    def test_initial_state(self):
        cpu = make_cpu(".word 0")
        self.assertEqual(cpu.pc, 0x00040000)
        self.assertEqual(cpu.regs, [0] * 32)
        self.assertEqual(cpu.run(), (0x00040000, 0))
        self.assertEqual(cpu.executed, 0)
```

```
    def test_registers_hold_unsigned_32bit(self):
        cpu = make_cpu("addi x1, x0, -1₩nsub x2, x0, x1₩nadd x3, x1, x1₩n.word 0")
        cpu.run()
        self.assertEqual(cpu.regs[1], 0xFFFFFFFF)
        self.assertEqual(cpu.regs[2], 1)
        self.assertEqual(cpu.regs[3], 0xFFFFFFFE)
        self.assertTrue(all(0 <= r <= 0xFFFFFFFF for r in cpu.regs))

    def test_blt_spec_question(self):
        """명세 9.1: 0xffffffff 와 1 비교 -> blt 는 분기한다 (-1 < 1)."""
        cpu = make_cpu("addi x1, x0, -1₩naddi x2, x0, 1₩nblt x1, x2, 8₩n"
                       "addi x3, x0, 1₩naddi x4, x0, 1₩n.word 0")
        cpu.run()
        self.assertEqual(cpu.regs[1], 0xFFFFFFFF)
        self.assertEqual((cpu.regs[3], cpu.regs[4]), (0, 1))   # x3 건너뜀

    def test_branch_target_is_relative_to_branch_pc(self):
        cpu = make_cpu("nop₩nbeq x0, x0, 8₩n.word 0₩n.word 0")
        self.assertEqual(cpu.run(), (0x0004000C, 0))   # 0x40004 + 8, PC+4 기준이 아님

    def test_termination_pc_is_fetch_pc(self):
        cpu = make_cpu("nop₩nnop₩n.word 0xffffffff₩n.word 0")
        self.assertEqual(cpu.run(), (0x00040008, 0xFFFFFFFF))
        self.assertEqual(cpu.executed, 2)

    def test_limit_boundary(self):
        # 정상 명령어 3개 후 invalid: 한도 3 -> 정상 종료
        cpu = make_cpu("nop₩nnop₩nnop₩n.word 0")
        self.assertEqual(cpu.run(limit=3), (0x0004000C, 0))
        # 정상 명령어 4개 후 invalid: 한도 3 -> 4번째(0x4000c) 실행 전 비정상 종료
        cpu = make_cpu("nop₩nnop₩nnop₩nnop₩n.word 0")
        with self.assertRaises(ExecutionLimitExceeded) as ctx:
            cpu.run(limit=3)
        self.assertEqual(ctx.exception.pc, 0x0004000C)
        self.assertEqual(cpu.executed, 3)

    def test_lw_rd_equals_rs1(self):
        cpu = make_cpu("lui x1, 0x10000₩nlw x1, 0(x1)₩n.word 0", data=bytes.fromhex("efbeadde"))
        cpu.run()
        self.assertEqual(cpu.regs[1], 0xDEADBEEF)


class OutputFormatTest(unittest.TestCase):
    def test_format(self):
        self.assertEqual(format_termination_line(0x00040014, 0),
                         "PC 0x00040014 invalid instruction 0x00000000₩n")
        self.assertEqual(format_termination_line(0x00040000, 0xFFFFFFFF),
                         "PC 0x00040000 invalid instruction 0xffffffff₩n")
        self.assertEqual(format_termination_line(0x0004ABC0, 0x0000006F),
                         "PC 0x0004abc0 invalid instruction 0x0000006f₩n")


if __name__ == "__main__":
    unittest.main(verbosity=1)
```

tests/mutation_test.py (155줄)

```
#!/usr/bin/env python3
"""변이 테스트(mutation test): 테스트 모음이 흔한 구현 실수를 실제로 잡아내는지 확인한다.

시뮬레이터 소스를 임시 디렉터리에 복사한 뒤 '일부러 버그를 하나 넣은 버전(변이)'을 만들고,
tests/cases 의 종단 간 테스트를 돌린다. 하나 이상 실패하면 그 변이는 '검출됨(killed)'.
검출되지 않은 변이가 있으면 테스트가 그 실수를 놓친다는 뜻이므로 테스트를 보강해야 한다.

사용법:  python tests/mutation_test.py [--markdown 결과.md]
"""

import argparse
import os
import shutil
import sys
import tempfile
from concurrent.futures import ThreadPoolExecutor

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, HERE)

from run_tests import CASES_DIR, run_case  # noqa: E402

# (설명, 파일, 원래 코드, 바꾼 코드)  - 원래 코드는 파일에 정확히 존재해야 한다(전부 교체)
MUTANTS = [
    ("blt 를 부호 없는 비교로 구현", "rvsim/cpu.py",
     "if to_signed32(rs1) < to_signed32(rs2):", "if rs1 < rs2:"),
    ("bge 를 부호 없는 비교로 구현", "rvsim/cpu.py",
     "if to_signed32(rs1) >= to_signed32(rs2):", "if rs1 >= rs2:"),
    ("bne 조건 반전", "rvsim/cpu.py",
     "if rs1 != rs2:", "if rs1 == rs2:"),
    ("분기 목표를 PC+4 기준으로 계산", "rvsim/cpu.py",
     "next_pc = pc + imm", "next_pc = pc + 4 + imm"),
    ("x0 쓰기 보호 누락", "rvsim/cpu.py",
     "if index != 0:", "if True:"),
    ("레지스터 값을 32비트로 자르지 않음", "rvsim/cpu.py",
     "self.regs[index] = to_u32(value)", "self.regs[index] = value"),
    ("andi/ori 즉시값을 부호 확장하지 않음(하위 12비트만)", "rvsim/cpu.py",
     "to_u32(imm)", "(imm & 0xFFF)"),
    ("lui 값을 한 번 더 12비트 시프트", "rvsim/cpu.py",
     "self.write_reg(inst.rd, imm)               # imm", "self.write_reg(inst.rd, imm << 12)  # imm"),
    ("lw 에서 오프셋 무시", "rvsim/cpu.py",
     "addr = to_u32(rs1 + imm)\n            self.write_reg",
     "addr = to_u32(rs1)\n            self.write_reg"),
    ("sw 가 rs2 대신 rs1 을 저장", "rvsim/cpu.py",
     "self.memory.store_word(addr, rs2)", "self.memory.store_word(addr, rs1)"),
    ("종료 시 fetch PC 대신 PC+4 출력", "rvsim/cpu.py",
     "return fetch_pc, word", "return fetch_pc + 4, word"),
    ("실행 한도 비교를 > 로 (1,000,001 개 허용)", "rvsim/cpu.py",
     "if self.executed >= limit:", "if self.executed > limit:"),
    ("실행 한도 검사를 invalid 판별보다 먼저 수행", "rvsim/cpu.py",
     "            if inst is None:                        # 지원하지 않는 명령어 -> 실행 없이 종료\n"
     "                return fetch_pc, word\n"
     "            if self.executed >= limit:              # 이미 한도만큼 실행했는데 또 정상 명령어\n"
     "                raise ExecutionLimitExceeded(fetch_pc, word)\n",
     "            if self.executed >= limit:\n"
     "                raise ExecutionLimitExceeded(fetch_pc, word)\n"
     "            if inst is None:\n"
     "                return fetch_pc, word\n"),
    ("I형 즉시값 부호 확장 누락", "rvsim/decoder.py",
     "return sign_extend(get_bits(word, 31, 20), 12)", "return get_bits(word, 31, 20)"),
    ("S형 즉시값 imm[4:0] 누락", "rvsim/decoder.py",
     "imm = (get_bits(word, 31, 25) << 5) | get_bits(word, 11, 7)",
     "imm = (get_bits(word, 31, 25) << 5)"),
    ("B형 즉시값 imm[11](bit 7) 누락", "rvsim/decoder.py",
     "| (get_bits(word, 7, 7) << 11)", "| (0 << 11)"),
    ("B형 즉시값을 12비트로 부호 확장", "rvsim/decoder.py",
     "return sign_extend(imm, 13)", "return sign_extend(imm, 12)"),
    ("R형 funct7 검사 누락 (sub 외에는 add 로 간주)", "rvsim/decoder.py",
     "R_TYPE_TABLE.get((funct3, funct7))",
     "R_TYPE_TABLE.get((funct3, funct7 if funct7 == 0b0100000 else 0))"),
    ("sub 를 add 로 해독", "rvsim/decoder.py",
     '(0b000, 0b0100000): "sub"', '(0b000, 0b0100000): "add"'),
    ("lw 의 funct3 검사 누락", "rvsim/decoder.py",
```

```
     "name = LOAD_TABLE.get(funct3)", 'name = "lw"'),
    ("sw 의 funct3 검사 누락", "rvsim/decoder.py",
     "name = STORE_TABLE.get(funct3)", 'name = "sw"'),
    ("I형 연산 funct3 검사 누락 (모르는 funct3 를 addi 로)", "rvsim/decoder.py",
     "name = OP_IMM_TABLE.get(funct3)", 'name = OP_IMM_TABLE.get(funct3, "addi")'),
    ("bltu/bgeu 를 blt/bge 로 해독", "rvsim/decoder.py",
     "name = BRANCH_TABLE.get(funct3)", "name = BRANCH_TABLE.get(funct3 & 0b101)"),
    ("little-endian 대신 big-endian 으로 읽기", "rvsim/memory.py",
     "return b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)",
     "return b3 | (b2 << 8) | (b1 << 16) | (b0 << 24)"),
    ("data.out 을 텍스트 모드로 저장", "rvsim/memory.py",
     'with open(path, "wb") as f:₩n        f.write(bytes(memory.data[0:DATA_SIZE]))',
     'with open(path, "w", encoding="latin-1") as f:₩n'
     '        f.write(bytes(memory.data[0:DATA_SIZE]).decode("latin-1"))'),
    ("data.out 을 1,020 바이트만 저장", "rvsim/memory.py",
     "bytes(memory.data[0:DATA_SIZE])", "bytes(memory.data[0:DATA_SIZE - 4])"),
    ("표준 출력을 텍스트 모드 print 로 출력", "sim.py",
     'sys.stdout.buffer.write(format_termination_line(pc, word).encode("ascii"))',
     'sys.stdout.write(format_termination_line(pc, word)); sys.stdout.flush()'),
    ("16진수를 대문자로 출력", "sim.py",
     '"PC 0x%08x invalid instruction 0x%08xWWn"', '"PC 0x%08X invalid instruction 0x%08XWWn"'),
    ("16진수 선행 0 생략", "sim.py",
     '"PC 0x%08x invalid instruction 0x%08xWWn"', '"PC 0x%x invalid instruction 0x%xWWn"'),
]


def make_mutant(workdir, rel_path, old, new):
    shutil.copy(os.path.join(ROOT, "sim.py"), workdir)
    shutil.copytree(os.path.join(ROOT, "rvsim"), os.path.join(workdir, "rvsim"),
                    ignore=shutil.ignore_patterns("__pycache__"))
    path = os.path.join(workdir, rel_path)
    with open(path, encoding="utf-8") as f:
        src = f.read()
    if old not in src:
        raise SystemExit("mutation pattern not found in %s:₩n%s" % (rel_path, old))
    with open(path, "w", encoding="utf-8", newline="") as f:
        f.write(src.replace(old, new))
    return os.path.join(workdir, "sim.py")


def run_suite(sim_cmd, names):
    with ThreadPoolExecutor(max_workers=8) as pool:
        results = list(pool.map(lambda n: (n, run_case(sim_cmd, os.path.join(CASES_DIR, n))),
                                names))
    return [n for n, problems in results if problems]


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--markdown", help="write result table to this markdown file")
    args = parser.parse_args()

    names = sorted(n for n in os.listdir(CASES_DIR) if os.path.isdir(os.path.join(CASES_DIR, n)))

    # 원본은 모든 테스트를 통과해야 의미가 있다.
    original_failures = run_suite([sys.executable, os.path.join(ROOT, "sim.py")], names)
    if original_failures:
        sys.exit("original simulator fails: %s" % original_failures)

    rows, survived = [], 0
    for desc, rel_path, old, new in MUTANTS:
        with tempfile.TemporaryDirectory() as work:
            sim_path = make_mutant(work, rel_path, old, new)
            failing = run_suite([sys.executable, sim_path], names)
        status = "검출" if failing else "미검출"
        survived += not failing
        shown = ", ".join(failing[:4]) + (" 외 %d개" % (len(failing) - 4) if len(failing) > 4 else "")
        print("[%s] %-40s %s" % (status, desc, shown))
        rows.append((desc, rel_path, len(failing), shown))

    print("₩n%d/%d mutants killed" % (len(MUTANTS) - survived, len(MUTANTS)))
    if args.markdown:
        with open(args.markdown, "w", encoding="utf-8", newline="₩n") as f:
            f.write("| 심은 버그 | 파일 | 실패한 테스트 수 | 실패한 테스트(일부) |₩n|---|---|---|---|₩n")
            for desc, rel_path, n, shown in rows:
                f.write("| %s | %s | %d | %s |₩n" % (desc, rel_path, n, shown or "**없음**"))
```

```
    sys.exit(1 if survived else 0)


if __name__ == "__main__":
    main()
```

tests/diff_ref_unicorn.py (199줄)

```
#!/usr/bin/env python3
"""(선택) 참조 실행기 Unicorn Engine 과의 차등 테스트 - 테스트 전용, 시뮬레이터 실행에는 쓰지 않는다.

Unicorn Engine(https://www.unicorn-engine.org, QEMU 기반 CPU 에뮬레이터)은 우리 팀이 작성하지 않은
독립 구현이다. 명세 8절에 따라 '참조 실행기를 별도 테스트용으로' 사용하며 보고서에 밝힌다.
시뮬레이터(sim.py, rvsim/)는 Unicorn 을 전혀 사용하지 않는다.

  1) tests/cases 의 정상 종료 케이스: 손으로 도출한 기대 data.out 을 Unicorn 실행 결과와 대조
     (기대 결과 자체가 맞는지 독립적으로 재확인)
  2) 무작위 프로그램 N 개: 우리 CPU 와 Unicorn 의 최종 레지스터 32개 · 데이터 1,024 바이트 · 종료 PC 비교

Unicorn 은 RV32 전체를 실행하므로 invalid instruction 에서 스스로 멈추지 않는다.
그래서 종료 지점(기대 출력의 PC)을 until 주소로 주어 그 직전에서 멈추게 한다.

사용법:  pip install unicorn
         python tests/diff_ref_unicorn.py [--programs 3000] [--seed 1]
"""

import argparse
import os
import random
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(HERE))
sys.path.insert(0, HERE)

try:
    from unicorn import UC_ARCH_RISCV, UC_MODE_RISCV32, Uc, UcError
    from unicorn.riscv_const import UC_RISCV_REG_PC, UC_RISCV_REG_X0
except ImportError:
    sys.exit("unicorn is not installed. run: pip install unicorn")

import asm  # noqa: E402
from rvsim.cpu import CPU  # noqa: E402
from rvsim.memory import Memory  # noqa: E402

TEXT_BASE, DATA_BASE = 0x00040000, 0x10000000
CASES_DIR = os.path.join(HERE, "cases")


def run_unicorn(text, data, stop_pc, max_count=2000000):
    """Unicorn 으로 stop_pc 에 도달할 때까지 실행. (레지스터 목록, 데이터 1,024 바이트) 반환.
    stop_pc 에 도달하지 못하면(다른 곳에서 멈추거나 예외) None 을 반환한다."""
    uc = Uc(UC_ARCH_RISCV, UC_MODE_RISCV32)
    text_size = (len(text) + 0xFFF) & ~0xFFF
    uc.mem_map(TEXT_BASE, max(text_size, 0x1000))
    uc.mem_map(DATA_BASE, 0x1000)
    uc.mem_write(TEXT_BASE, text)
    uc.mem_write(DATA_BASE, (data + bytes(1024))[:1024])
    try:
        uc.emu_start(TEXT_BASE, stop_pc, count=max_count)
    except UcError:
        return None
    if uc.reg_read(UC_RISCV_REG_PC) & 0xFFFFFFFF != stop_pc:
        return None
    regs = [uc.reg_read(UC_RISCV_REG_X0 + i) & 0xFFFFFFFF for i in range(32)]
    return regs, bytes(uc.mem_read(DATA_BASE, 1024))


def read_bytes(path):
    with open(path, "rb") as f:
        return f.read()


def check_hand_expectations():
    """정상 종료 케이스의 손 계산 기대값 == Unicorn 결과 인지 확인."""
    checked, bad = 0, 0
    for name in sorted(os.listdir(CASES_DIR)):
        d = os.path.join(CASES_DIR, name)
        if not os.path.isdir(d):
            continue
        if int(read_bytes(os.path.join(d, "expected_exit_code.txt"))) != 0:
            continue
```

```
        read = lambda fn: read_bytes(os.path.join(d, fn))  # noqa: E731
        stop_pc = int(read("expected_stdout.txt").split()[1], 16)
        result = run_unicorn(read("text.bin"), read("data.bin"), stop_pc)
        ok = result is not None and result[1] == read("expected_data.out")
        checked += 1
        bad += not ok
        print("%s  %s" % ("OK  " if ok else "DIFF", name))
    return checked, bad


# ---------------------------------------------------------------------------
# 무작위 프로그램 생성
#   x31 = 0x10000000, x30 = 0x10000400 : lw/sw 기준 주소 (변경 금지)
#   x29 = 루프 카운터 (루프 블록 전용)
#   분기는 앞으로만(같은 블록 안 또는 블록 끝) -> 반드시 종료
# ---------------------------------------------------------------------------
SPECIAL = [0, 1, 0xFFFFFFFF, 0x80000000, 0x7FFFFFFF, 0xFFFFF800, 0x800, 0x7FF]


def rand_imm12(rng):
    return rng.choice([-2048, -1, 0, 1, 2047, rng.randint(-2048, 2047), rng.randint(-16, 16)])


def rand_mem(rng):
    if rng.random() < 0.5:
        return "%d(x31)" % (4 * rng.randint(0, 255))
    return "%d(x30)" % (-4 * rng.randint(1, 256))


def rand_simple(rng):
    """분기가 아닌 명령어 하나 (rd 는 x0~x28)."""
    rd, rs1, rs2 = rng.randint(0, 28), rng.randint(0, 31), rng.randint(0, 31)
    kind = rng.random()
    if kind < 0.35:
        return "%s x%d, x%d, x%d" % (rng.choice(["add", "sub", "and", "or"]), rd, rs1, rs2)
    if kind < 0.65:
        return "%s x%d, x%d, %d" % (rng.choice(["addi", "andi", "ori"]), rd, rs1, rand_imm12(rng))
    if kind < 0.75:
        return "lui x%d, 0x%x" % (rd, rng.getrandbits(20))
    if kind < 0.87:
        return "lw x%d, %s" % (rd, rand_mem(rng))
    return "sw x%d, %s" % (rs2, rand_mem(rng))


def random_program(rng):
    lines = ["lui x31, 0x10000", "lui x30, 0x10000", "addi x30, x30, 1024"]
    for r in range(1, 29):                      # 레지스터 초기값 무작위
        v = rng.choice(SPECIAL) if rng.random() < 0.3 else rng.getrandbits(32)
        hi, lo = (v + 0x800) >> 12 & 0xFFFFF, ((v & 0xFFF) ^ 0x800) - 0x800
        lines += ["lui x%d, 0x%x" % (r, hi), "addi x%d, x%d, %d" % (r, r, lo)]

    label = [0]

    def new_label():
        label[0] += 1
        return "L%d" % label[0]

    for _ in range(rng.randint(2, 6)):
        if rng.random() < 0.3:                  # 루프 블록
            top = new_label()
            lines.append("addi x29, x0, %d" % rng.randint(1, 5))
            body = [rand_simple(rng) for _ in range(rng.randint(1, 8))]
            lines.append("%s: %s" % (top, body[0]))
            lines += body[1:]
            lines += ["addi x29, x29, -1", "bne x29, x0, %s" % top]
        else:                                   # 직선 블록 + 앞으로 가는 분기
            n = rng.randint(5, 30)
            labels = [new_label() for _ in range(n + 1)]   # labels[n] = 블록 끝
            block = []
            for i in range(n):
                if rng.random() < 0.25:
                    j = rng.randint(i + 1, n)
                    block.append("%s x%d, x%d, %s" % (rng.choice(["beq", "bne", "blt", "bge"]),
                                                      rng.randint(0, 31), rng.randint(0, 31),
                                                      labels[j]))
                else:
```

```
                    block.append(rand_simple(rng))
            for i in range(n):
                lines.append("%s: %s" % (labels[i], block[i]))
            lines.append("%s: addi x0, x0, 0" % labels[n])
    lines.append(".word 0")
    return "\n".join(lines)


def check_random(count, seed):
    rng = random.Random(seed)
    for k in range(count):
        src = random_program(rng)
        words = asm.assemble(src)
        text = asm.words_to_bytes(words)
        data = bytes(rng.getrandbits(8) for _ in range(rng.choice([0, 16, 1024])))
        end_pc = TEXT_BASE + 4 * (len(words) - 1)

        cpu = CPU(Memory(text, data))
        pc, word = cpu.run()
        result = run_unicorn(text, data, end_pc)
        uc_regs, uc_data = result if result is not None else ([None] * 32, None)

        if (pc, word) != (end_pc, 0) or cpu.regs != uc_regs or bytes(cpu.memory.data) != uc_data:
            print("MISMATCH in random program #%d (seed %d)" % (k, seed))
            if result is None:
                print("  unicorn did not reach end pc 0x%08x" % end_pc)
            for i in range(32):
                if result is not None and cpu.regs[i] != uc_regs[i]:
                    print("  x%d: ours 0x%08x, unicorn 0x%08x" % (i, cpu.regs[i], uc_regs[i]))
            print(src)
            return False
    return True


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--programs", type=int, default=3000)
    parser.add_argument("--seed", type=int, default=1)
    args = parser.parse_args()

    checked, bad = check_hand_expectations()
    print("hand-derived expectations: %d/%d agree with unicorn" % (checked - bad, checked))
    ok = check_random(args.programs, args.seed)
    print("random differential test: %d programs %s" % (args.programs, "OK" if ok else "FAILED"))
    sys.exit(0 if ok and not bad else 1)


if __name__ == "__main__":
    main()
```

tools/trace.py (103줄)

```
#!/usr/bin/env python3
"""디버깅 보조 도구: 명령어 단위 실행 추적과 레지스터 덤프 (채점 대상 출력이 아님).

sim.py 와 같은 rvsim 코드로 실행하면서, 명령어마다
  실행 순번, PC, 기계어, 디스어셈블 결과, 바뀐 레지스터/메모리
를 출력하고, 마지막에 레지스터 32개를 덤프한다. data.out 은 만들지 않는다.

사용법
  python tools/trace.py text.bin data.bin             # 전체 추적
  python tools/trace.py text.bin data.bin --limit 50  # 처음 50개만 추적 출력
  python tools/trace.py text.bin --disasm             # text.bin 디스어셈블만
"""

import argparse
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from rvsim.cpu import CPU, ExecutionLimitExceeded  # noqa: E402
from rvsim.decoder import decode  # noqa: E402
from rvsim.memory import TEXT_BASE, Memory, SimError, read_binary_file  # noqa: E402


def disassemble(inst, pc):
    n = inst.name
    if n in ("add", "sub", "and", "or"):
        return "%-4s x%d, x%d, x%d" % (n, inst.rd, inst.rs1, inst.rs2)
    if n in ("addi", "andi", "ori"):
        return "%-4s x%d, x%d, %d" % (n, inst.rd, inst.rs1, inst.imm)
    if n == "lui":
        return "lui  x%d, 0x%x" % (inst.rd, inst.imm >> 12)
    if n == "lw":
        return "lw   x%d, %d(x%d)" % (inst.rd, inst.imm, inst.rs1)
    if n == "sw":
        return "sw   x%d, %d(x%d)" % (inst.rs2, inst.imm, inst.rs1)
    return "%-4s x%d, x%d, %d   # -> 0x%08x" % (n, inst.rs1, inst.rs2, inst.imm,
                                               (pc + inst.imm) & 0xFFFFFFFF)


class TracingCPU(CPU):
    def __init__(self, memory, show_limit):
        super().__init__(memory)
        self.show_limit = show_limit

    def execute(self, inst, pc):
        if self.executed >= self.show_limit:      # 출력하지 않는 구간은 스냅샷 없이 실행만
            return super().execute(inst, pc)
        before_regs = list(self.regs)
        before_data = bytes(self.memory.data)
        next_pc = super().execute(inst, pc)
        changes = ["x%d = 0x%08x" % (i, self.regs[i]) for i in range(32)
                   if self.regs[i] != before_regs[i]]
        for off in range(0, len(before_data), 4):
            if self.memory.data[off:off + 4] != before_data[off:off + 4]:
                value = int.from_bytes(self.memory.data[off:off + 4], "little")
                changes.append("[0x%08x] = 0x%08x" % (0x10000000 + off, value))
        if inst.name in ("beq", "bne", "blt", "bge"):
            changes.append("taken" if next_pc != pc + 4 else "not taken")
        word = self.memory.fetch(pc)
        print("#%-7d 0x%08x  0x%08x  %-34s %s" % (self.executed + 1, pc, word,
                                                  disassemble(inst, pc), "  ".join(changes)))
        return next_pc


def dump_registers(regs):
    for row in range(8):
        print("  ".join("x%-2d = 0x%08x" % (i, regs[i]) for i in range(row, 32, 8)))


def main():
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("text")
```

```
    parser.add_argument("data", nargs="?")
    parser.add_argument("--limit", type=int, default=10 ** 9, help="trace only first N")
    parser.add_argument("--disasm", action="store_true", help="only disassemble text.bin")
    args = parser.parse_args()

    text = read_binary_file(args.text)
    if args.disasm:
        mem = Memory(text, b"")
        for off in range(0, len(text) - len(text) % 4, 4):
            pc = TEXT_BASE + off
            word = mem.fetch(pc)
            inst = decode(word)
            print("0x%08x  0x%08x  %s" % (pc, word,
                                          disassemble(inst, pc) if inst else "(invalid)"))
        return

    data = read_binary_file(args.data) if args.data else b""
    cpu = TracingCPU(Memory(text, data), args.limit)
    try:
        pc, word = cpu.run()
        print("STOP    0x%08x  0x%08x  invalid instruction (executed %d)" % (pc, word, cpu.executed))
    except (ExecutionLimitExceeded, SimError) as e:
        print("ERROR   %s (executed %d)" % (e, cpu.executed))
    print("Wnregisters:")
    dump_registers(cpu.regs)


if __name__ == "__main__":
    main()
```

report/build_report.py (156줄)

```
#!/usr/bin/env python3
"""report.md -> report.html -> report.pdf 변환 (보고서 작성용 보조 도구, 시뮬레이터와 무관).

필요한 것: pip install markdown, 그리고 Microsoft Edge 또는 Google Chrome/Chromium (PDF 인쇄용)

  python report/build_report.py                # report.pdf 생성
  python report/build_report.py --html-only    # report.html 만 생성 (브라우저에서 열어 인쇄해도 됨)
  python report/build_report.py --browser "C:/경로/chrome.exe"

삽입 규칙
  <!-- MUTATION_TABLE --> : report/mutation_results.md 내용
                            (python tests/mutation_test.py --markdown report/mutation_results.md 로 갱신)
  <!-- TEAM_PHOTO -->     : report/team_photo.jpg (또는 .jpeg/.png) 가 있으면 사진, 없으면 안내 상자
"""

import argparse
import os
import re
import shutil
import subprocess
import sys

try:
    import markdown
except ImportError:
    sys.exit("python-markdown is required: pip install markdown")

HERE = os.path.dirname(os.path.abspath(__file__))

CSS = """
@page { size: A4; margin: 18mm 16mm 18mm 16mm;
        @bottom-center { content: counter(page) " / " counter(pages); font-size: 9pt; color: #666; } }
body { font-family: "Malgun Gothic", "Apple SD Gothic Neo", "Noto Sans KR", sans-serif;
       font-size: 10pt; line-height: 1.55; color: #111; }
h1 { font-size: 19pt; border-bottom: 2px solid #2b4a6f; padding-bottom: 4px; color: #1d3557; }
h2 { font-size: 14pt; color: #1d3557; border-bottom: 1px solid #c8d3e0; padding-bottom: 2px;
     margin-top: 22px; break-after: avoid; }
h3 { font-size: 11.5pt; color: #2b4a6f; margin-top: 14px; break-after: avoid; }
table { border-collapse: collapse; width: 100%; margin: 8px 0 12px; font-size: 8.8pt;
        break-inside: auto; }
tr { break-inside: avoid; }
th, td { border: 1px solid #9aa8b8; padding: 3px 6px; vertical-align: top; }
th { background: #e8eef6; }
code { font-family: Consolas, "D2Coding", monospace; font-size: 8.8pt; background: #f3f5f8;
       padding: 0 2px; border-radius: 2px; }
pre { background: #f6f8fa; border: 1px solid #d5dbe3; padding: 7px 9px; font-size: 8.6pt;
      line-height: 1.4; white-space: pre-wrap; break-inside: avoid; }
pre code { background: none; padding: 0; }
blockquote { border-left: 4px solid #e0a800; background: #fff8e1; margin: 8px 0; padding: 4px 10px; }
svg { display: block; margin: 8px auto; break-inside: avoid; }
.photo { width: 100%; max-height: 120mm; object-fit: contain; border: 1px solid #ccc; }
.placeholder { border: 2px dashed #b55; color: #b55; padding: 40px; text-align: center; }
"""

BROWSER_CANDIDATES = [
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
    "/Applications/Google Chrome.app/Contents/MacOS/Google Chrome",
    "/Applications/Microsoft Edge.app/Contents/MacOS/Microsoft Edge",
]


def find_browser(explicit):
    if explicit:
        return explicit
    for path in BROWSER_CANDIDATES:
        if os.path.exists(path):
            return path
    for name in ("msedge", "google-chrome", "chromium", "chromium-browser", "chrome"):
        found = shutil.which(name)
        if found:
            return found
```

```
    return None


def normalize_lists(text):
    """Python-Markdown 은 목록 앞에 빈 줄이 있어야 하고 중첩 목록은 4칸 들여쓰기를 요구한다.
    GitHub 식으로 쓴 report.md 를 그대로 변환할 수 있게 맞춰 준다(코드 블록은 건드리지 않음)."""
    out, in_code, prev = [], False, ""
    for line in text.split("₩n"):
        if line.lstrip().startswith("```"):
            in_code = not in_code
        elif not in_code:
            m = re.match(r"^( *)([-*]|₩d+₩.) ", line)
            if m:
                if len(m.group(1)) in (2, 3):
                    line = "    " + line.lstrip()
                prev_in_list = re.match(r"^ *([-*]|₩d+₩.) ", prev) or prev.startswith("  ")
                if prev.strip() and not prev_in_list:
                    out.append("")
        out.append(line)
        prev = line
    return "₩n".join(out)


def build_html():
    with open(os.path.join(HERE, "report.md"), encoding="utf-8") as f:
        text = normalize_lists(f.read())

    mutation_path = os.path.join(HERE, "mutation_results.md")
    if os.path.exists(mutation_path):
        with open(mutation_path, encoding="utf-8") as f:
            text = text.replace("<!-- MUTATION_TABLE -->", f.read())
    else:
        text = text.replace("<!-- MUTATION_TABLE -->",
                            "(python tests/mutation_test.py --markdown report/mutation_results.md 실행 필요)")

    photo = next((p for p in ("team_photo.jpg", "team_photo.jpeg", "team_photo.png")
                  if os.path.exists(os.path.join(HERE, p))), None)
    if photo:
        text = text.replace("<!-- TEAM_PHOTO -->",
                            '<img class="photo" src="%s" alt="팀 사진">' % photo)
    else:
        text = text.replace("<!-- TEAM_PHOTO -->",
                            '<div class="placeholder">【프로젝트 실행 중 팀원 전원이 함께 찍은 사진을 '
                            'report/team_photo.jpg 로 저장한 뒤 다시 빌드】</div>')

    md = markdown.Markdown(extensions=["tables", "fenced_code"])
    md.block_level_elements.append("svg")
    body = md.convert(text)
    html = ('<!DOCTYPE html>₩n<html lang="ko"><head><meta charset="utf-8">'
            "<title>RISC-V 시뮬레이터 보고서</title><style>%s</style></head><body>%s</body></html>"
            % (CSS, body))
    html_path = os.path.join(HERE, "report.html")
    with open(html_path, "w", encoding="utf-8") as f:
        f.write(html)
    return html_path


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--html-only", action="store_true")
    parser.add_argument("--browser")
    args = parser.parse_args()

    html_path = build_html()
    print("wrote", html_path)
    if args.html_only:
        return

    browser = find_browser(args.browser)
    if not browser:
        sys.exit("Edge/Chrome not found. open report.html in a browser and print to PDF, "
                 "or pass --browser PATH")
    pdf_path = os.path.join(HERE, "report.pdf")
    url = "file:///" + html_path.replace("₩₩", "/").lstrip("/")
    subprocess.run([browser, "--headless", "--disable-gpu", "--no-pdf-header-footer",
                    "--print-to-pdf-no-header", "--print-to-pdf=" + pdf_path, url],
```

```
                    check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=120)
    print("wrote", pdf_path)


if __name__ == "__main__":
    main()
```